



République Algérienne Démocratique et
Populaire

Ministère de l'Enseignement Supérieur et de la
Recherche Scientifique



Université des Sciences et de la Technologie Houari Boumediene

Faculté d'Informatique

Mémoire de Master

Filière : Informatique

Spécialité

Ingénierie des Logiciels

Thème

CONCEPTION ET RÉALISATION D'UNE PLATEFORME PAAS
POUR LE DÉPLOIEMENT AUTOMATIQUE D'APPLICATIONS WEB
AVEC INTÉGRATION GIT ET GESTION DE RESSOURCES CLOUD

Sujet encadré par :

Mlle Rihane Selma

Soutenu le : ../06/2026

Présenté par :

BOUCHAREB Rami

Devant le jury composé de :

.....

.....

Projet N° 000/2026

Remerciement

Tout d'abord je remercie bien évidemment Dieu le plus miséricordieux de m'avoir accordé la force et la patience pour réaliser ce projet de fin d'études. En second lieu, je tiens à exprimer toute ma gratitude envers mon encadrante dans ce mémoire, Mlle Rihane Selma, pour sa patience, sa disponibilité et surtout ses conseils avisés, qui ont guidé mon travail vers une conclusion réussie. Je tiens également à exprimer ma profonde gratitude envers le corps enseignant de mon université, car sans leur accompagnement, je n'aurais pas acquis toutes les compétences nécessaires à la réalisation de ce projet. Je souhaite également remercier les membres du jury d'avoir accepté d'évaluer ce travail modeste. Enfin, j'adresse mes plus sincères remerciements à tous mes proches, amis et famille, qui ont contribué de près ou de loin à l'achèvement de ce travail. Merci à tous et à toutes.

Résumé

Ce projet vise à concevoir et implémenter une plateforme PaaS innovante pour le déploiement en un clic d'applications web, en utilisant un lien GitHub public ou privé via une authentification OAuth sécurisée. Le déploiement s'effectue dans un environnement cloud orchestré par Kubernetes, qui gère l'orchestration des conteneurs Docker pour assurer scalabilité, haute disponibilité (HA) avec auto-scaling horizontal basé sur les métriques de charge, et une stratégie multi-tenancy adoptant l'approche silo pour isoler complètement les ressources des utilisateurs, garantissant ainsi une sécurité renforcée et une confidentialité des données. L'expérience utilisateur est centrée sur une interface web intuitive et responsive, permettant non seulement de soumettre le dépôt Git pour un build automatique, mais aussi de monitorer en temps réel les logs d'exécution, les métriques de performance (CPU, mémoire, trafic réseau), et de générer dynamiquement des sous-domaines avec activation automatique de certificats SSL via Let's Encrypt pour une accessibilité sécurisée. De plus, la plateforme intègre une gestion avancée des environnements, incluant la configuration de variables d'environnement (env) et de secrets sensibles (comme les clés API ou mots de passe) stockés de manière chiffrée dans Kubernetes Secrets, évitant toute exposition accidentelle. Un historique des déploiements est maintenu, offrant des fonctionnalités de rollback vers des versions précédentes en cas d'erreur, avec un journal détaillé des builds, tests et mises en production pour une traçabilité complète. La persistance des volumes est assurée via des PersistentVolumes dans Kubernetes, permettant aux applications stateful (comme celles avec bases de données) de conserver les données malgré les redéploiements ou scaling. Inspiré de solutions comme Railway, Render ou Vercel, ce système supporte divers langages et frameworks (Node.js, Python, etc.), intègre des pipelines CI/CD automatisés pour des mises à jour continues via webhooks GitHub, et favorise une adoption agile dans des contextes DevOps, réduisant les temps de mise en production tout en optimisant les coûts cloud via une allocation dynamique des ressources, rendant la plateforme adaptée à des environnements hybrides ou multi-cloud.

Mots Clée : PaaS, Déploiement automatique, GitHub, Git, Cloud computing, Kubernetes, Haute disponibilité, SSL.

Table des matières

Table des figures	v
Liste des tableaux	viii
Introduction Générale	1
Structure Du Mémoire	2
1 État de l’Art et Analyse de l’Existant	4
1.1 Le Cloud Computing et les modèles de service	4
1.1.1 Présentation des modèles IaaS, PaaS, SaaS	4
1.2 L’orchestration de conteneurs — Docker et Kubernetes	6
1.2.1 La conteneurisation avec Docker	6
1.2.2 Kubernetes comme standard d’orchestration	7
1.3 Analyse des plateformes PaaS internationales	9
1.3.1 Étude comparative des leaders mondiaux	9
1.4 Analyse des initiatives PaaS algériennes	11
1.4.1 Étude critique des acteurs locaux émergents	11
1.5 Le contexte réglementaire algérien	12
1.5.1 Lois 18-07 et 25-11	12
1.5.2 Obligations pour l’hébergement de données	12
1.5.3 Impact sur les startups et freelances	12
1.6 Analyse du marché et identification du besoin	12
1.6.1 Quantification du marché cible	12
1.6.2 Identification des pain points	12
1.7 Synthèse et positionnement de Hostifer	12
1.7.1 Lacunes identifiées	12
1.7.2 Positionnement unique	12

2	Conception et Architecture du Système	14
2.1	Recueil et spécification des besoins	14
2.1.1	Identification des acteurs	14
2.1.2	Besoins fonctionnels	18
2.1.3	Besoins non fonctionnels	21
2.2	Modélisation UML des cas d'utilisation	24
2.2.1	Cas d'utilisation détaillés	24
2.2.2	Diagramme de cas d'utilisation global	25
2.2.3	Cas d'utilisation détaillés	25
2.3	Modélisation des classes et données	25
2.3.1	Diagramme de classes	25
2.3.2	Modèle Entité-Association	25
2.3.3	Description des entités principales	25
2.4	Modélisation comportementale	32
2.4.1	Diagrammes de séquence	32
2.4.2	Diagrammes d'activité	32
2.4.3	Diagramme d'états	32
2.5	Architecture technique globale	32
2.5.1	Vue d'ensemble de l'architecture	32
2.5.2	Architecture multi-tenant	32
2.5.3	Architecture du pipeline de build	32
2.5.4	Architecture de collecte des logs	32
2.5.5	Architecture de gestion des secrets	32
2.6	Choix technologiques	32
2.6.1	Justification des technologies retenues	32
3	Réalisation et Implémentation	38
3.1	Environnement de développement et de déploiement	38
3.1.1	Infrastructure matérielle	38
3.1.2	Outils de développement	41
3.1.3	Organisation des dépôts GitHub	43
3.2	Implémentation du backend NestJS	45
3.2.1	Structure modulaire de l'application	45
3.2.2	Module d'authentification	46
3.2.3	Module de déploiement et orchestration Argo Workflows	48
3.2.4	Module de streaming des logs	50
3.2.5	Module de gestion des variables d'environnement avec Vault	53

3.2.6	Versioning et rollback des déploiements	53
3.3	Implémentation du builder Go	53
3.3.1	Structure du binaire	53
3.3.2	Détection de framework	55
3.3.3	Génération du plan Railpack	56
3.3.4	Construction et push de l'image via Buildkit	59
3.4	Implémentation de l'infrastructure Kubernetes	60
3.4.1	Chart Helm multi-tenant	61
3.4.2	Pipeline Argo Workflows	65
3.4.3	Configuration de l'observabilité (Loki + Promtail)	67
3.4.4	Gestion des certificats TLS avec cert-manager	67
3.4.5	Collecte des métriques avec Prometheus	67
3.5	Présentation des interfaces utilisateur	67
3.5.1	Page d'accueil (Landing Page)	67
3.5.2	Tableau de bord utilisateur	68
3.5.3	Assistant de déploiement (Deploy Wizard)	68
3.5.4	Gestion des variables d'environnement	68
3.5.5	Interface d'administration Argo Workflows	68
3.6	Tests et validation	68
3.6.1	Tests fonctionnels	68
3.6.2	Tests d'isolation multi-tenant	68
3.6.3	Tests de charge et performance	68
3.6.4	Tests de conformité réglementaire	68
3.7	Bilan et perspectives	68
3.7.1	Fonctionnalités réalisées	68
3.7.2	Difficultés rencontrées	68
3.7.3	Perspectives d'évolution	68
	Conclusion Générale	70
	Bibliographie	71

Table des figures

1.1	Pyramide des modèles de service cloud (IaaS / PaaS / SaaS) avec exemples de plateformes pour chaque niveau	6
1.2	Architecture d'un cluster Kubernetes (Master node, Worker nodes, etcd, API Server)	8
1.3	Cycle de vie d'un Pod Kubernetes (Pending → Running → Succeeded/Failed)	9
1.4	Diagramme TAM/SAM/SOM du marché algérien du cloud pour développeurs	12
1.5	Matrice de positionnement compétitif (axes : qualité technique / conformité algérienne)	13
2.1	Diagramme de cas d'utilisation global montrant tous les acteurs et leurs interactions avec le système (authentification, déploiement, gestion des projets, administration, CI/CD, logs)	33
2.2	Diagramme de classes UML complet (User, Project, Deployment, DeploymentStep, EnvVar, Plan, GitHubInstallation, GitHubRepository, ActivityLog, Notification, Domain)	34
2.3	Diagramme Entité-Association de la base de données PostgreSQL avec cardinalités	35
2.4	Diagramme de séquence : Authentification (NextJS → NestJS → PostgreSQL → JWT)	36
2.5	Diagramme de séquence : Déclenchement d'un déploiement (Frontend → NestJS → Argo Workflows → Build Job → Registry → Helm → DNS) . .	36
2.6	Diagramme de séquence : Streaming des logs en temps réel (Builder Pod → Promtail → Loki → NestJS SSE → Browser)	36
2.7	Diagramme de séquence : Webhook CI/CD GitHub Push (GitHub → NestJS → DeploymentsService → Argo Workflows)	36
2.8	Diagramme d'activité du pipeline de déploiement complet (Build → Provision → DNS → Live)	36

2.9	Diagramme d'activité de la détection de framework et génération du plan Railpack	36
2.10	Diagramme d'états d'un Deployment (QUEUED → BUILDING → PROVISIONING → CONFIGURING_DNS → LIVE / FAILED / CANCELLED)	36
2.11	Schéma d'architecture générale de Hostifer : couche client (NextJS), couche API (NestJS), couche orchestration (Argo Workflows), couche infrastructure (Kubernetes, Buildkit, Registry, Loki, Vault), couche réseau (Traefik, Cloudflare, cert-manager)	37
2.12	Schéma d'isolation multi-tenant : Namespace par projet, NetworkPolicy bloquant le trafic inter-tenant, ressources quotas par plan	37
2.13	Schéma du pipeline de build (Job Kubernetes Go binary → Railpack CLI → Buildkitd Deployment → Registry interne)	37
2.14	Schéma de la chaîne de collecte et streaming des logs (Pod stdout → Promtail DaemonSet → Loki → SSE → Browser)	37
2.15	Schéma de gestion des secrets avec HashiCorp Vault et External Secrets Operator	37
3.1	Schéma d'organisation des dépôts GitHub et registres OCI	45
3.2	Arborescence du projet NestJS avec description de chaque module (Auth, Users, Projects, Deployments, Logs, EnvVars, GitHub, Plans, Infrastructure)	45
3.3	Extrait du plan railpack-plan.json généré pour une application Node.js avec les modifications appliquées	58
3.4	Extrait du template NetworkPolicy illustrant l'isolation inter-tenant et l'autorisation Traefik uniquement	65
3.5	Capture d'écran de l'interface Argo Workflows UI montrant un workflow de déploiement complet avec les trois étapes	67
3.6	Capture d'écran de la section Hero de la landing page	67
3.7	Capture d'écran de la section Pricing	67
3.8	Capture d'écran de la section comparaison concurrentielle	68
3.9	Capture d'écran du dashboard avec liste des projets et statuts	68
3.10	Capture d'écran de la page de détail d'un projet	69
3.11	Capture d'écran de l'étape 1 : validation de l'URL GitHub et détection du framework	69
3.12	Capture d'écran de l'étape 2 : configuration (région, plan, variables d'environnement)	69
3.13	Capture d'écran de l'étape 3 : streaming des logs de build en temps réel . .	69

3.14	Capture d'écran de l'étape 4 : confirmation du déploiement réussi avec URL live	69
3.15	Capture d'écran de l'interface de gestion des variables d'environnement avec masquage des secrets	69
3.16	Capture d'écran de l'interface Argo Workflows montrant les étapes Build, Provision, DNS d'un déploiement réel	69
3.17	Graphique du temps de déploiement moyen par framework (de la soumission du workflow à l'état LIVE)	69

Liste des tableaux

1.1	Comparaison des responsabilités client/fournisseur dans les trois modèles .	5
1.2	Comparaison des plateformes PaaS internationales (hébergement, prix, frameworks supportés, CI/CD, free tier)	11
1.3	Comparaison des plateformes PaaS algériennes (fonctionnalités, infrastructure réelle, état de maturité)	11
1.4	Synthèse des obligations légales imposées par les lois 18-07 et 25-11 et leur impact sur l'hébergement cloud	12
1.5	Synthèse des besoins non satisfaits et réponse apportée par Hostifer	12
2.1	Synthèse des acteurs du système Hostifer	17
2.2	Tableau des besoins fonctionnels	19
2.3	Tableau des besoins non fonctionnels	22
2.4	Tableau récapitulatif des cas d'utilisation (ID, nom, acteur principal, priorité)	25
2.5	Description de l'entité User	26
2.6	Description de l'entité Project	27
2.7	Description de l'entité Deployment	29
2.8	Description de l'entité User (attributs, types, contraintes)	31
2.9	Description de l'entité Project	31
2.10	Description de l'entité Deployment	31
2.11	Ressources allouées par profil d'allocation (CPU request/limit, RAM request/limit, stockage, pods max). Les métriques d'autoscaling supportées et les comportements HPA/VPA autorisés par profil sont indiqués dans la grille	32
2.12	Tableau des choix technologiques (composant, technologie retenue, alternatives considérées, justification du choix)	32
3.1	Caractéristiques de l'environnement de déploiement	40
3.2	Environnement de développement	42
3.3	Organisation des dépôts GitHub et artefacts GHCR	45

3.4	Correspondance entre les phases Argo Workflows et les statuts internes Hostifer	50
3.5	Structure du projet builder Go	55
3.6	Ressources Kubernetes provisionnées par le chart tenant-chart	63
3.7	Résultats des tests de déploiement par framework (Express, NestJS, Next.js, Flask, Laravel) : statut, durée de build, mémoire consommée	69
3.8	Résultats des tests d'isolation réseau (tentative de connexion inter-tenant, résultat attendu vs observé)	69
3.9	Tableau de performance : temps de build moyen, latence SSE, consom- mation mémoire buildkitd pour 1, 3 et 5 builds simultanés, plus métriques d'autoscaling (temps de montée HPA, recommandations VPA, latence d'ajout de nœuds)	69
3.10	Checklist de conformité (critère, mécanisme technique de conformité, statut vérifié)	69
3.11	Tableau de couverture fonctionnelle (besoin identifié au chapitre 2, statut d'implémentation : réalisé / partiel / reporté)	69

Introduction Générale

À l'ère de la transformation numérique accélérée, l'infrastructure cloud est devenue le socle indispensable de tout projet logiciel moderne. Les développeurs, startups et entreprises du monde entier s'appuient désormais sur des plateformes de déploiement automatisé pour mettre leurs applications en production rapidement, de manière fiable et sans maîtrise approfondie des mécanismes d'infrastructure sous-jacents. Des solutions de déploiement cloud ont démocratisé ce paradigme à l'échelle internationale, offrant une expérience de déploiement fluide résumée en quelques clics.

Cependant, l'écosystème numérique algérien se heurte à des obstacles structurels qui rendent ces plateformes internationales difficilement accessibles. Les cartes bancaires algériennes sont fréquemment rejetées sur les plateformes étrangères, les frais de change imposent une surcharge financière significative, et la latence introduite par des serveurs situés en Europe ou aux États-Unis dégrade l'expérience des utilisateurs finaux locaux. Plus grave encore, les législations algériennes en vigueur — notamment la loi 18-07 relative à la protection des personnes physiques dans le traitement des données à caractère personnel, et la loi 25-11 portant sur la souveraineté numérique — imposent des contraintes réglementaires croissantes sur l'hébergement des données à l'étranger, exposant les entreprises locales à des risques de conformité non négligeables.

C'est dans ce contexte que s'inscrit **Hostifer**, la première plateforme algérienne de type *Platform-as-a-Service* (PaaS) reposant sur une architecture Kubernetes native. Hostifer a pour vocation de combler le fossé technologique entre les développeurs algériens et les standards internationaux de déploiement cloud, en proposant une solution entièrement hébergée en Algérie, facturée en dinars algériens (DZD), et conçue pour être conforme aux exigences réglementaires nationales.

La plateforme permet à tout développeur, qu'il soit étudiant, freelance ou membre d'une startup, de déployer une application web en moins de cinq minutes à partir d'une simple URL GitHub, sans aucune connaissance préalable en administration système ou en orchestration de conteneurs. Le processus est entièrement automatisé : détection du framework applicatif, construction de l'image Docker via Railpack et Buildkit, provision-

nement d'un environnement Kubernetes isolé, configuration du certificat SSL, et création d'un sous-domaine dédié — le tout orchestré par Argo Workflows et supervisé en temps réel par un système de streaming de logs basé sur Loki et Promtail.

Notre projet vise ainsi à répondre à plusieurs enjeux simultanément :

- **Un enjeu technique** : concevoir une architecture multi-tenant robuste, isolée et scalable, exploitant les primitives natives de Kubernetes telles que les *Namespaces*, les *NetworkPolicies*, les *ResourceQuotas* et les *LimitRanges*, pour garantir une isolation stricte entre les projets des différents utilisateurs.
- **Un enjeu économique** : proposer une tarification transparente en dinars algériens, compatible avec les moyens de paiement locaux (carte CIB, Edahabia, BaridiMob, virement bancaire), et offrir un palier gratuit pour favoriser l'adoption auprès des développeurs étudiants et indépendants.
- **Un enjeu réglementaire** : assurer la conformité avec les lois algériennes sur la protection des données et la souveraineté numérique, en hébergeant l'intégralité de l'infrastructure sur le cloud algérien DjazzyCloud, opéré en partenariat avec Alibaba Cloud.
- **Un enjeu de souveraineté numérique** : contribuer à l'émergence d'une infrastructure cloud locale de qualité professionnelle, réduisant la dépendance des acteurs numériques algériens vis-à-vis des hébergeurs étrangers.

Structure Du Mémoire

Ce mémoire détaille la conception, le développement et la mise en œuvre de la plateforme Hostifer. Il s'articule autour d'une introduction générale, de trois chapitres principaux, d'une conclusion générale et d'une bibliographie.

— CHAPITRE 1

Nous dressons un état de l'art des plateformes PaaS existantes, en analysant les solutions internationales leaders du marché ainsi que les initiatives locales algériennes émergentes. Cette analyse comparative nous permet d'identifier les lacunes du marché algérien, de justifier la pertinence du projet Hostifer, et de définir les exigences fonctionnelles et non fonctionnelles auxquelles la plateforme doit répondre.

— **CHAPITRE 2**

Nous présentons la phase de conception de la plateforme, en décrivant les choix architecturaux retenus : l'architecture multi-tenant sur Kubernetes, le pipeline de déploiement orchestré par Argo Workflows, la gestion des secrets via HashiCorp Vault, et la stratégie de collecte et de streaming des logs applicatifs. Des diagrammes UML et des schémas d'architecture accompagnent cette description pour en illustrer les interactions et les flux de données.

— **CHAPITRE 3**

Nous abordons la phase de réalisation, en détaillant les technologies utilisées pour le développement du backend (NestJS), du frontend (Next.js), et de l'infrastructure (k3s, Traefik, cert-manager, Buildkit, Railpack, Loki, Promtail). Nous présentons les interfaces de la plateforme, le processus de déploiement de bout en bout, ainsi que les résultats des tests fonctionnels réalisés sur des applications représentatives de différents frameworks supportés.

Chapitre 1

État de l'Art et Analyse de l'Existant

1.1 Le Cloud Computing et les modèles de service

1.1.1 Présentation des modèles IaaS, PaaS, SaaS

Le cloud computing repose sur trois grands modèles de service qui se distinguent par la répartition des responsabilités entre le fournisseur d'infrastructure et le client. Ces modèles constituent une progression logique allant de la simple mise à disposition de ressources brutes jusqu'à la consommation d'une plateforme de déploiement entièrement gérée.

Infrastructure as a Service (IaaS) Le modèle IaaS fournit au client les ressources matérielles virtualisées essentielles : machines virtuelles, stockage, réseau et pare-feu. Le fournisseur administre l'infrastructure physique, tandis que le client conserve la maîtrise du système d'exploitation, du middleware, des runtime applicatifs et des applications elles-mêmes. Ce modèle offre une forte flexibilité, mais demande également davantage de compétences d'administration.

Platform as a Service (PaaS) Le modèle PaaS se situe à un niveau d'abstraction supérieur. Le fournisseur prend en charge non seulement l'infrastructure, mais aussi une grande partie de la pile logicielle nécessaire au déploiement : environnement d'exécution, orchestration, mise à l'échelle, supervision et parfois intégration continue. Le développeur se concentre alors sur le code applicatif, la configuration du projet et les variables d'environnement. Le PaaS réduit fortement la complexité opérationnelle et accélère le passage du développement à la production.

Software as a Service (SaaS) Le modèle SaaS correspond à l'exposition d'une application complète prête à l'emploi. L'utilisateur final consomme directement le service via

une interface web ou une API, sans gérer ni l'infrastructure ni la plateforme sous-jacente. Ce modèle maximise la simplicité d'usage, mais laisse peu de contrôle sur l'environnement d'exécution et la personnalisation technique.

Différences essentielles La différence principale entre ces trois approches réside dans le niveau de responsabilité confié au client. En IaaS, le client gère encore une large part de l'exploitation technique. En PaaS, il se concentre surtout sur l'application. En SaaS, il n'administre plus l'environnement de déploiement et se limite à l'utilisation du service.

Positionnement de Hostifer Hostifer s'inscrit clairement dans la catégorie PaaS. La plateforme ne se contente pas de fournir des serveurs virtuels : elle automatise le déploiement d'applications à partir d'un dépôt GitHub, orchestre la construction des images, provisionne l'environnement Kubernetes, configure le routage réseau et expose l'application en ligne. L'utilisateur n'a pas à gérer les nœuds, les pods, les manifests Kubernetes ou les détails du runtime. Son rôle est centré sur le projet applicatif, la configuration du build et les secrets nécessaires au déploiement. Cette approche correspond précisément à la promesse d'un PaaS moderne.

Cas d'usage L'IaaS est adapté aux équipes qui souhaitent un contrôle fin de leur infrastructure ou qui exécutent des charges de travail très spécifiques. Le PaaS convient aux développeurs et aux startups qui veulent publier rapidement une application web sans mobiliser une équipe d'exploitation dédiée. Le SaaS vise les utilisateurs qui cherchent un service immédiatement utilisable, comme une messagerie, un CRM ou un outil de collaboration.

TABLEAU 1.1 – Comparaison des responsabilités client/fournisseur dans les trois modèles

Critère	IaaS	PaaS	SaaS
Infrastructure physique	Fournisseur	Fournisseur	Fournisseur
Système d'exploitation	Client	Fournisseur	Fournisseur
Plateforme d'exécution / middle-ware	Client	Fournisseur	Fournisseur
Application	Client	Client	Fournisseur
Données et configuration métier	Client	Client	Client / partiellement fournisseur selon le service

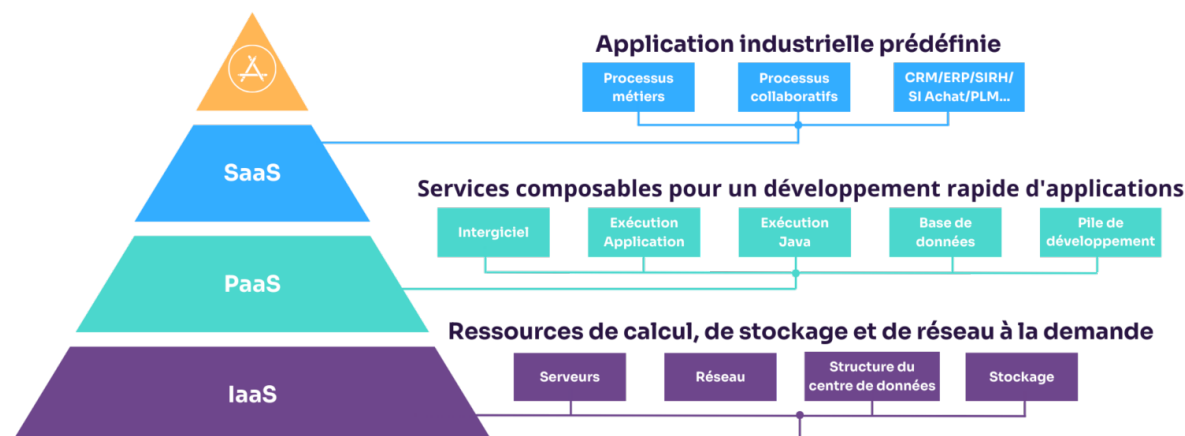


FIGURE 1.1 – Pyramide des modèles de service cloud (IaaS / PaaS / SaaS) avec exemples de plateformes pour chaque niveau

1.2 L'orchestration de conteneurs — Docker et Kubernetes

1.2.1 La conteneurisation avec Docker

La conteneurisation est une approche qui permet d'encapsuler une application avec toutes ses dépendances dans une unité légère, portable et isolée appelée conteneur. Contrairement à une machine virtuelle, un conteneur partage le noyau du système hôte tout en maintenant un environnement d'exécution cohérent pour l'application. Cette propriété facilite le déploiement, réduit les écarts entre les environnements de développement et de production, et améliore la reproductibilité des applications.

Docker s'est imposé comme l'outil de référence pour la création, l'exécution et la distribution de conteneurs. Il permet de définir une image à partir d'un *Dockerfile*, de construire cette image de manière reproductible, puis de lancer un ou plusieurs conteneurs à partir de celle-ci. Dans un contexte applicatif moderne, Docker constitue donc la brique de base qui transforme une application en artefact déployable.

Cependant, à mesure que le nombre de conteneurs augmente, leur gestion manuelle devient difficile. Il devient nécessaire d'automatiser le déploiement, la montée en charge, la mise à jour progressive, la résilience et l'exposition réseau des applications. C'est pré-

cisément ce rôle qu'assure Kubernetes.

1.2.2 Kubernetes comme standard d'orchestration

Kubernetes est aujourd'hui le standard dominant pour l'orchestration de conteneurs. Il fournit un plan de contrôle capable de piloter un ensemble de nœuds de travail, de planifier les conteneurs, de maintenir l'état désiré des applications et de les faire évoluer automatiquement selon la charge. Là où Docker se concentre sur l'emballage et l'exécution d'un conteneur, Kubernetes gère le cycle de vie complet d'applications conteneurisées à grande échelle.

Pod Le Pod est l'unité de base de déploiement dans Kubernetes. Il regroupe un ou plusieurs conteneurs partageant le même réseau, le même stockage temporaire et le même cycle de vie. Dans la majorité des cas, un Pod contient un seul conteneur applicatif, mais le modèle permet aussi d'associer des conteneurs auxiliaires pour des tâches de supervision ou de synchronisation.

Namespace Un Namespace permet de segmenter logiquement les ressources du cluster. Il sert à isoler des environnements, des équipes ou des clients différents au sein d'une même infrastructure Kubernetes. Dans un contexte multi-tenant, le Namespace constitue une première couche d'isolation organisationnelle et opérationnelle.

Deployment Le Deployment décrit l'état souhaité d'une application et pilote la création, la mise à jour et le redémarrage des Pods associés. Il permet notamment les mises à jour progressives, les retours arrière et la gestion déclarative du nombre de réplicas. Pour une plateforme PaaS, le Deployment est essentiel car il automatise l'exécution stable des applications déployées.

Service Un Service fournit un point d'accès stable vers un ensemble de Pods dont l'adresse peut changer au fil du temps. Il joue un rôle de façade réseau et permet la découverte de service à l'intérieur du cluster ou depuis l'extérieur, selon le type de Service utilisé. Il découple ainsi les consommateurs d'une application de l'instabilité des Pods sous-jacents.

Ingress L'Ingress définit des règles d'exposition HTTP/HTTPS vers les services internes du cluster. Il permet de router les requêtes en fonction du nom de domaine, du

chemin d'URL ou d'autres critères applicatifs. Associé à un contrôleur Ingress, il constitue la couche d'entrée réseau principale pour exposer les applications web de manière contrôlée.

ResourceQuota Le ResourceQuota fixe des limites globales sur les ressources consommées dans un Namespace : CPU, mémoire, nombre de Pods, volume de stockage ou autres objets Kubernetes. Il empêche qu'un projet n'épuise les capacités du cluster et permet de faire respecter les contraintes d'un plan d'abonnement ou d'un tenant donné.

NetworkPolicy La NetworkPolicy définit les règles de communication réseau autorisées entre Pods, Namespaces ou sélecteurs précis. Elle permet de bloquer les flux non désirés et d'appliquer une véritable isolation réseau entre clients ou services. Dans une architecture multi-tenant, elle est indispensable pour garantir qu'un projet ne puisse pas communiquer avec les ressources d'un autre projet sans autorisation explicite.

L'association de ces primitives permet de construire une plateforme d'hébergement moderne, automatisée et sécurisée. Kubernetes fournit l'abstraction nécessaire pour déployer, mettre à l'échelle et isoler les applications, tandis que Docker fournit le format d'exécution portable sur lequel repose la distribution des workloads.

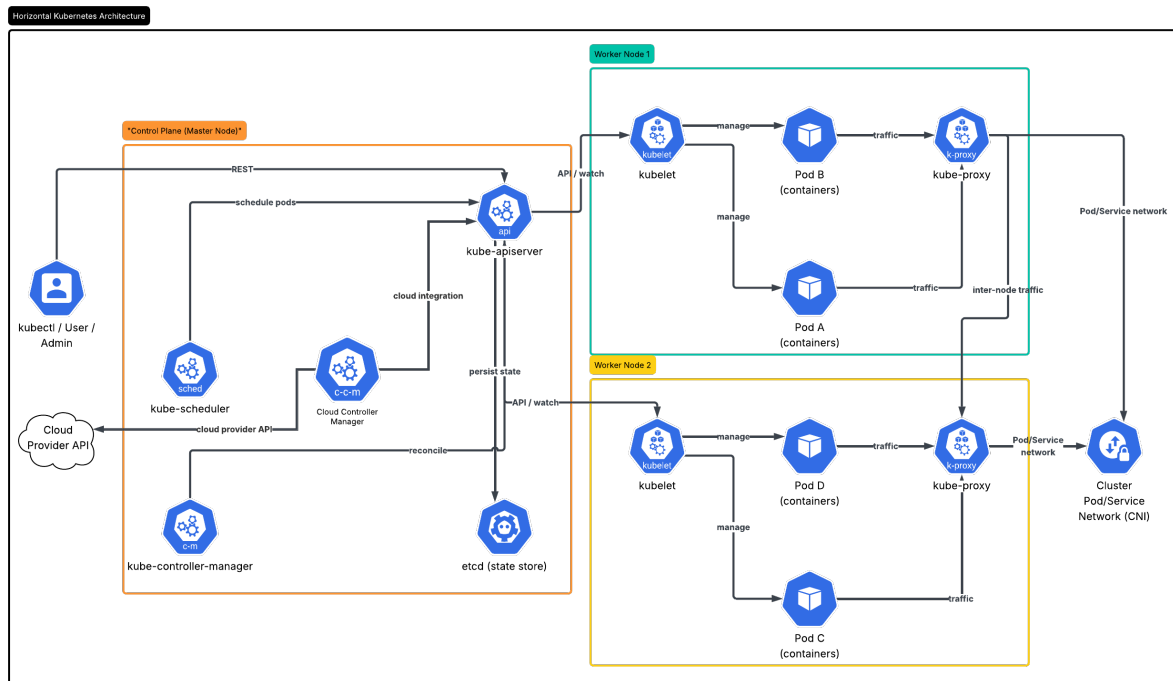


FIGURE 1.2 – Architecture d'un cluster Kubernetes (Master node, Worker nodes, etcd, API Server)

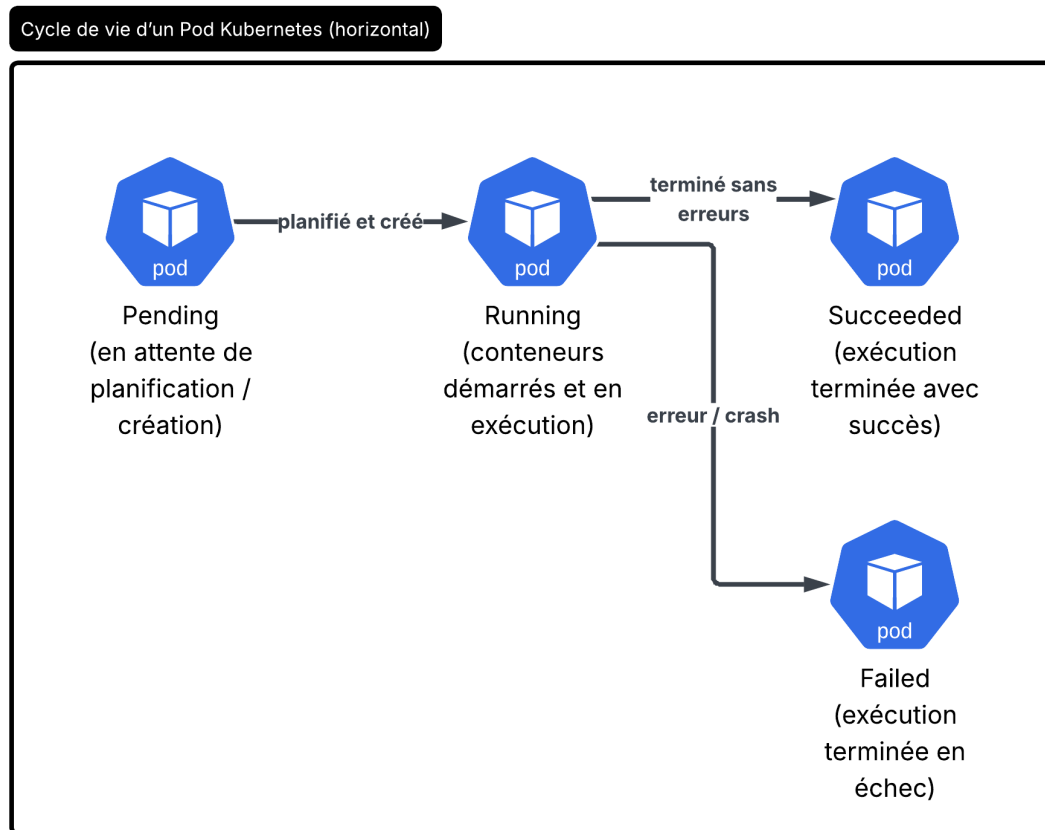


FIGURE 1.3 – Cycle de vie d'un Pod Kubernetes (Pending → Running → Succeeded/Failed)

1.3 Analyse des plateformes PaaS internationales

1.3.1 Étude comparative des leaders mondiaux

Pour bien comprendre le positionnement de Hostifer, il est essentiel d'examiner les plateformes PaaS internationales qui dominent le marché. Ces solutions se sont établies comme les références dans le déploiement d'applications web et cloud-natives. Nous analysons ici six acteurs majeurs, chacun avec sa spécialité, ses forces et ses limitations particulières pour les développeurs algériens.

Vercel Vercel s'est construit autour de Next.js et excelle dans le déploiement d'applications frontend modernes et de Full-Stack JavaScript. Son modèle repose sur un git push automatisant le déploiement sur un CDN global distribué. Les technologies sous-jacentes incluent Cloudflare, Node.js et des runtimes propriétaires. La tarification suit le modèle pay-as-you-go basé sur les requêtes HTTP et le stockage. Vercel brille par sa simplicité et

son intégration GitHub, mais elle demeure limitée aux écosystèmes Node.js/JavaScript. Limitation Algérie : pas de support des cartes bancaires locales, pas de serveurs en Afrique du Nord, latence importante pour les utilisateurs finaux algériens.

Railway Railway propose un modèle friendly où l'utilisateur pousse simplement du code et la plateforme détecte le langage et le framework. Elle supporte Node.js, Python, Go, Rust et bien d'autres. L'infrastructure repose sur Kubernetes, Nixpacks pour la détection, et Docker. La tarification est basée sur la consommation de ressources (CPU, mémoire, bande passante). Railway offre une excellente expérience développeur et un free tier généreux, mais reste orientée vers les petits projets et startups. Limitation Algérie : pas de région serveur locale, frais de change élevés, difficulté d'accès au paiement pour les startups algériennes sans compte bancaire international.

Render Render cible les développeurs cherchant simplicité et pricing transparent. Il supporte Node.js, Python, Go, Rust et offre un déploiement depuis Git avec reconstruction automatique. L'architecture s'appuie sur Docker et une orchestration propre. Les tarifs sont clairs et prévisibles, basés sur les ressources et la durée. Render fournit un free tier limité et des plans payants progressifs. Limitation Algérie : pas de infrastructure locale, carte de crédit internationale obligatoire, latence réseau vers l'Algérie non optimisée.

Heroku Heroku a longtemps été l'incontournable du PaaS grâce à sa simplicité (git push to deploy) et sa polyvalence. Heroku supporte une vingtaine de langages et frameworks. L'infrastructure repose sur AWS avec une couche d'abstraction propriétaire. La tarification était autrefois très accessible, mais Salesforce a drastiquement augmenté les coûts en 2022, rendant la plateforme désormais très chère pour les développeurs individuels et startups. Limitation Algérie : coûts prohibitifs, pas de data center régional, support de paiement limité aux carte étrangères.

Fly.io Fly.io innove avec son modèle de déploiement distribué : déployer une application dans plusieurs régions du monde simultanément pour minimiser la latence. Fly supporte Docker et tout langage/framework. L'infrastructure utilise une orchestration Kubernetes propriétaire. Les tarifs offrent un free tier modeste et un pay-as-you-go ensuite basé sur les ressources. Fly est idéal pour les applications distribuées, mais manque de datacenter en Afrique. Limitation Algérie : pas de point de présence régional (le plus proche est en Europe), latence toujours présente, infrastructure non-souveraine.

Koyeb Koyeb est une plateforme émergente française mettant l'accent sur la simplicité et la performance. Elle supporte Docker, serverless et une détection automatique des frameworks. Infrastructure sur AWS Europe. Tarification pay-as-you-go avec free tier. Koyeb cherche à se positionner comme alternative européenne simplifiée à Vercel/Netlify. Limitation Algérie : aucun datacenter en Afrique, fondée récemment donc moins mature que les autres, support utilisateur limité, frais bancaires internationaux.

Synthèse et limitations communes Toutes ces plateformes partagent plusieurs limitations majeures pour le contexte algérien. Premièrement, aucune n'opère d'infrastructure en Algérie ou en Afrique du Nord, forçant une latence significative et une dépendance vis-à-vis d'infrastructures étrangères. Deuxièmement, aucune n'accepte les cartes bancaires algériennes (CIB, EDAHABIA), limitant l'accès aux développeurs locaux. Troisièmement, la tarification en devises étrangères avec frais de change rend ces services plus chers pour les utilisateurs algériens. Enfin, aucune ne respecte explicitement la législation algérienne sur la souveraineté numérique (loi 25-11).

TABEAU 1.2 – Comparaison des plateformes PaaS internationales (hébergement, prix, frameworks supportés, CI/CD, free tier)

1.4 Analyse des initiatives PaaS algériennes

1.4.1 Étude critique des acteurs locaux émergents

TABEAU 1.3 – Comparaison des plateformes PaaS algériennes (fonctionnalités, infrastructure réelle, état de maturité)

TABLEAU 1.4 – Synthèse des obligations légales imposées par les lois 18-07 et 25-11 et leur impact sur l'hébergement cloud

1.5 Le contexte réglementaire algérien

1.5.1 Lois 18-07 et 25-11

1.5.2 Obligations pour l'hébergement de données

1.5.3 Impact sur les startups et freelances

1.6 Analyse du marché et identification du besoin

1.6.1 Quantification du marché cible

1.6.2 Identification des pain points

FIGURE 1.4 – Diagramme TAM/SAM/SOM du marché algérien du cloud pour développeurs

TABLEAU 1.5 – Synthèse des besoins non satisfaits et réponse apportée par Hostifer

1.7 Synthèse et positionnement de Hostifer

1.7.1 Lacunes identifiées

1.7.2 Positionnement unique

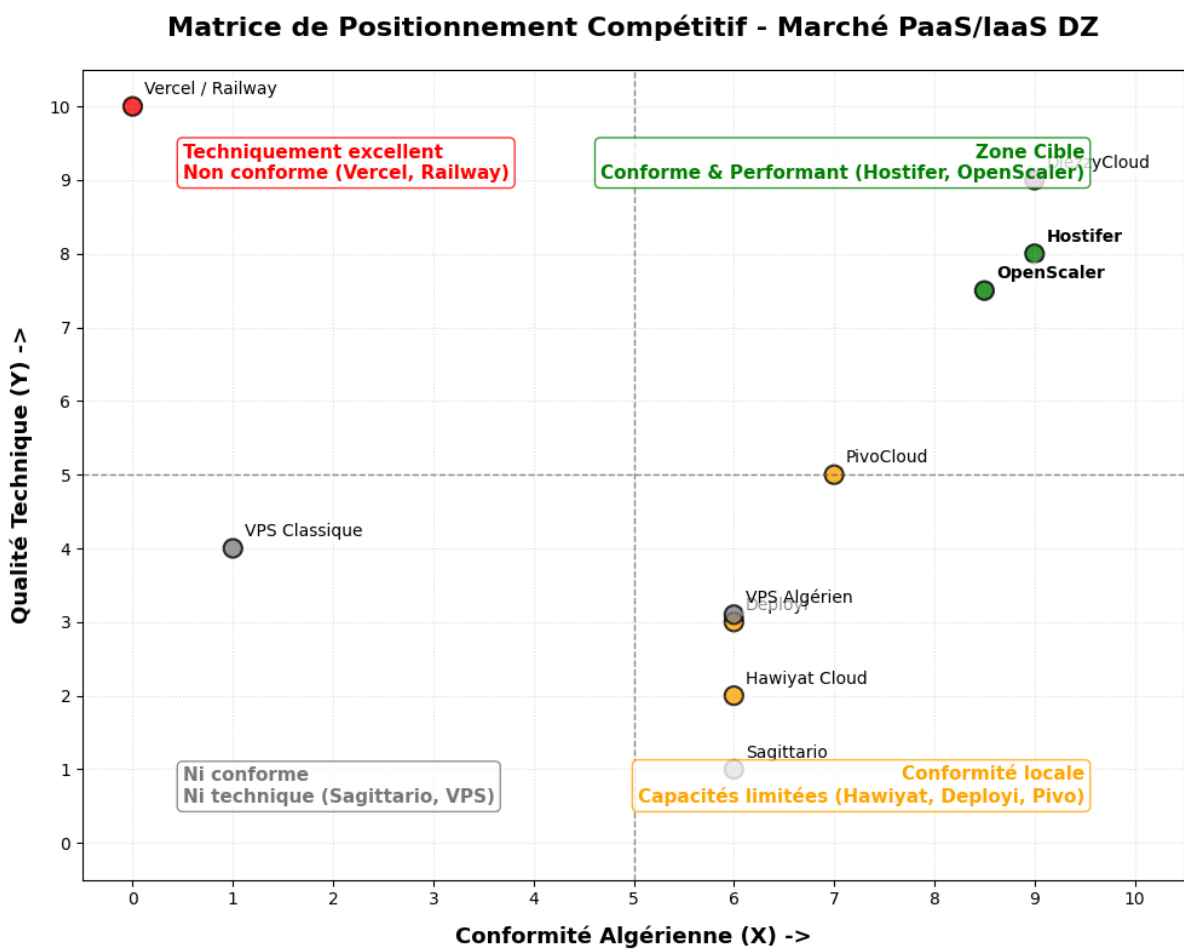


FIGURE 1.5 – Matrice de positionnement compétitif (axes : qualité technique / conformité algérienne)

Chapitre 2

Conception et Architecture du Système

2.1 Recueil et spécification des besoins

La phase de recueil et de spécification des besoins constitue le fondement de toute démarche d'ingénierie logicielle rigoureuse. Elle permet d'établir un contrat clair entre les parties prenantes et l'équipe de développement, en formalisant les attentes fonctionnelles et les contraintes non fonctionnelles du système. Dans le cadre de Hostifer, cette phase revêt une importance particulière, car la plateforme doit répondre simultanément aux exigences d'une infrastructure multi-tenant à haute disponibilité, aux contraintes réglementaires algériennes, et aux attentes d'utilisabilité propres aux développeurs. Les besoins ont été recueillis par une analyse itérative combinant l'étude des solutions existantes (menée au chapitre précédent) et la modélisation des cas d'usage réels identifiés lors de la conception.

% —————

2.1.1 Identification des acteurs

Au sens de la norme UML 2.5, un acteur est un *behavioed classifier* qui modélise le rôle joué par une entité externe au système, qu'il s'agisse d'un utilisateur humain, d'un système logiciel tiers ou d'un composant matériel, et qui interagit avec le sujet par échange de signaux et de données [1]. L'acteur est, par définition, placé en dehors de la frontière du système ; il n'en fait pas partie mais en déclenche ou en consulte les comportements observables [2]. L'identification rigoureuse des acteurs est une étape préalable à toute modélisation des cas d'utilisation, car elle délimite le périmètre du système et constitue le point de départ de l'élicitation des besoins [3].

Pour Hostifer, l'analyse des diagrammes de séquence et des flux d'interaction a permis d'identifier deux catégories d'acteurs : les **acteurs humains**, qui initient des actions via

une interface utilisateur, et les **acteurs systèmes**, qui interagissent avec la plateforme de manière automatisée, soit en tant que systèmes externes déclencheurs, soit en tant que composants d'infrastructure participant aux workflows internes. Le tableau 2.1 présente la synthèse complète des acteurs identifiés.

Acteurs humains

L'Utilisateur (Développeur) est l'acteur principal de la plateforme. Il représente tout profil de développeur souhaitant déployer et opérer des applications web sans gérer directement l'infrastructure sous-jacente : ingénieur logiciel, freelance, membre d'une startup ou d'une PME. L'utilisateur interagit avec Hostifer exclusivement via le **frontend Next.js**, qui constitue le point d'entrée graphique de la plateforme. Ses actions couvrent l'ensemble du cycle de vie applicatif : authentification, création et configuration de projets, déclenchement de déploiements, consultation des journaux de build en temps réel, gestion des variables d'environnement, et activation du pipeline CI/CD automatique.

L'Administrateur de la plateforme est un acteur humain secondaire disposant de droits étendus sur l'ensemble du système. Il supervise la santé de l'infrastructure, gère les plans tarifaires et les quotas de ressources par tenant, et intervient en cas d'incident. L'administrateur interagit principalement avec les outils d'observabilité et d'orchestration (tableaux de bord Grafana, interface Argo Workflows UI) ainsi qu'avec les API d'administration exposées par le **backend NestJS**.

Acteurs systèmes externes

NextAuth.js est la bibliothèque d'authentification intégrée au frontend Next.js. Elle gère les sessions utilisateur côté client, orchestre le flux OAuth 2.0 vers GitHub, et délègue l'émission et le renouvellement des jetons JWT au backend NestJS via des callbacks configurables. Bien qu'elle soit un composant interne au frontend, son rôle de médiateur entre l'utilisateur, le fournisseur OAuth et le backend en fait un participant structurant des diagrammes d'interaction.

GitHub OAuth est le fournisseur d'identité externe utilisé pour l'authentification déléguée. Lors d'une connexion via OAuth 2.0, GitHub valide l'identité de l'utilisateur et retourne un code d'autorisation que NextAuth.js échange contre un jeton d'accès. En complément, **GitHub** agit comme système déclencheur du pipeline CI/CD : à chaque événement **push** sur une branche configurée, il émet une requête HTTP vers l'endpoint webhook du backend NestJS, initiant automatiquement un nouveau déploiement sans intervention humaine.

Acteurs systèmes d'orchestration et d'infrastructure

Argo Workflows est le moteur d'orchestration des pipelines de déploiement. Il reçoit les soumissions de *WorkflowTemplates* depuis le backend NestJS via l'API REST Argo, puis exécute séquentiellement les trois étapes du pipeline : *Build*, *Provision* et *DNS*. Argo Workflows interagit directement avec l'**API Kubernetes** pour créer et surveiller les ressources d'exécution de chaque étape.

Le Builder Job (Go) est le composant exécuté sous forme de Job Kubernetes lors de l'étape *Build*. Ce binaire Go orchestre la chaîne de construction complète : clonage du dépôt Git, invocation de **Railpack CLI** pour la détection du framework et la génération du plan de build, connexion au daemon **Buildkitd** pour la construction de l'image OCI, et push de l'image résultante vers le **registre interne (Registry OCI)**.

Helm est le gestionnaire de packages Kubernetes utilisé lors de l'étape *Provision*. Il déploie ou met à jour le chart Helm dédié à chaque tenant, créant l'ensemble des ressources Kubernetes nécessaires : *Namespace*, *Deployment*, *Service*, *NetworkPolicy*, *ResourceQuota*, *HorizontalPodAutoscaler* et *ExternalSecret*.

L'API Kubernetes est le point d'entrée central de l'infrastructure d'exécution. Elle est sollicitée par Argo Workflows, par Helm et directement par le backend NestJS pour la création, la mise à jour et la suppression de l'ensemble des ressources Kubernetes associées aux tenants [4].

Vault + ESO (External Secrets Operator) constituent le sous-système de gestion des secrets. HashiCorp Vault stocke les variables d'environnement sensibles des tenants dans un moteur KV v2 chiffré. L'External Secrets Operator synchronise ces secrets vers des ressources *Kubernetes Secret* natives, qui sont ensuite injectées dans les pods applicatifs via `envFrom`, sans que les valeurs ne transitent jamais en clair.

Traefik est le contrôleur d'entrée (*Ingress Controller*) du cluster. Il assure le routage du trafic HTTP/HTTPS entrant vers les services des tenants selon les règles d'Ingress-Route définies par le chart Helm, et s'intègre avec **cert-manager** pour le provisionnement et le renouvellement automatiques des certificats TLS via Let's Encrypt.

Promtail (DaemonSet) et **Loki** forment le sous-système de collecte et d'indexation des journaux. Promtail, déployé en tant que DaemonSet sur chaque nœud du cluster, collecte les flux `stdout` des pods de construction et y attache des labels structurés (`build_id`, `tenant_id`, `log_type`). Les journaux sont transmis à Loki pour indexation et persistance, depuis lesquels le backend NestJS les interroge via l'API `query_range` pour les diffuser en temps réel à l'utilisateur via SSE (*Server-Sent Events*).

TABLEAU 2.1 – Synthèse des acteurs du système Hostifer

Acteur	Type	Catégorie	Rôle principal
Utilisateur	Humain	Principal	Déploie et opère ses applications via le frontend Next.js
Administrateur	Humain	Secondaire	Supervise la plateforme et l'infrastructure
NextAuth.js	Système interne	Secondaire	Gère les sessions et orchestre le flux OAuth côté frontend
GitHub OAuth	Système externe	Secondaire	Fournisseur d'identité OAuth 2.0
GitHub (web-hook)	Système externe	Déclencheur	Déclenche le CI/CD via événement push
Argo Workflows	Système interne	Orchestrateur	Exécute les pipelines Build / Provision / DNS
Builder Job (Go)	Système interne	Automatisé	Construit et pousse l'image OCI via Railpack et Buildkitd
Railpack CLI	Système interne	Automatisé	Détecte le framework et génère le plan de build
Buildkitd	Système interne	Automatisé	Construit l'image OCI à partir du plan Railpack
Registry (OCI)	Système interne	Automatisé	Stocke les images OCI des applications
Kubernetes API	Système interne	Infrastructure	Point d'entrée pour toutes les opérations sur les ressources cluster
Helm	Système interne	Infrastructure	Provisionne les ressources Kubernetes par tenant via chart
Vault + ESO	Système interne	Sécurité	Stocke et injecte les secrets applicatifs chiffrés

Suite à la page suivante

Acteur	Type	Catégorie	Rôle principal
cert-manager	Système interne	Infrastructure	Émet et renouvelle les certificats TLS automatiquement
Traefik	Système interne	Infrastructure	Route le trafic entrant et assure la terminaison TLS
Loki	Système interne	Observabilité	Indexe et persiste les journaux des pods
Promtail (DaemonSet)	Système interne	Observabilité	Collecte les journaux des pods et les transmet à Loki

2.1.2 Besoins fonctionnels

Les besoins fonctionnels décrivent l'ensemble des services que le système doit fournir à ses utilisateurs ainsi que les comportements attendus en réponse à des entrées spécifiques [3]. Ils ont été structurés en sept domaines fonctionnels cohérents avec les grands cas d'utilisation de la plateforme, et priorisés selon la méthode MoSCoW (*Must have, Should have, Could have, Won't have*) [5], qui permet d'aligner les efforts de développement sur les fonctionnalités à plus fort impact métier.

Authentification et gestion des comptes. La plateforme doit permettre à tout utilisateur de s'inscrire et de s'authentifier, soit par une paire identifiant/mot de passe avec hachage sécurisé, soit par le protocole OAuth 2.0 délégué à GitHub. Le système doit émettre et renouveler des jetons d'accès JWT (*JSON Web Token*) de manière transparente, avec un mécanisme de rotation des jetons de rafraîchissement afin de maintenir la sécurité des sessions longues.

Gestion des projets. Un développeur authentifié doit pouvoir créer un projet en fournissant l'URL d'un dépôt GitHub, configurer ses paramètres (région de déploiement, plan tarifaire associé), consulter la liste de ses projets avec leur statut en temps réel, et supprimer un projet avec déprovisionnement complet des ressources Kubernetes associées.

Déploiement d'applications. La fonctionnalité centrale de la plateforme consiste à déployer automatiquement une application depuis un dépôt GitHub vers l'infrastructure Kubernetes. Ce processus doit couvrir la détection automatique du framework utilisé, la construction de l'image conteneur via Railpack et Buildkit, son stockage dans un registre interne, et son déploiement via un chart Helm dédié par tenant. L'utilisateur doit avoir accès à l'historique complet des déploiements et pouvoir effectuer un retour arrière vers une version stable antérieure.

Journalisation en temps réel. Pendant et après le processus de construction, l'utilisateur doit pouvoir consulter les journaux de build en continu, sans rechargement de page, via une connexion SSE (*Server-Sent Events*). Les journaux doivent être indexés et persistés dans Loki afin d'être consultables a posteriori.

Gestion des variables d'environnement. La plateforme doit permettre la création, la mise à jour et la suppression de variables d'environnement associées à chaque projet. Les valeurs sensibles (secrets) doivent être stockées de manière chiffrée dans HashiCorp Vault et jamais exposées en clair dans l'interface ou dans les journaux.

Intégration et livraison continues (CI/CD). Le système doit permettre la configuration d'un pipeline CI/CD automatique, déclenché par un événement `push` GitHub via webhook. L'utilisateur doit pouvoir activer ou désactiver ce comportement par projet, et choisir la branche de déploiement cible.

Administration de la plateforme. L'administrateur doit disposer d'une interface permettant de visualiser l'ensemble des tenants actifs, leur consommation de ressources, et les workflows Argo Workflows en cours d'exécution. Il doit pouvoir intervenir sur les ressources Kubernetes directement depuis les outils d'observabilité intégrés.

Le tableau 2.2 synthétise l'ensemble des besoins fonctionnels avec leur identifiant, leur description, leur priorité MoSCoW et l'acteur concerné.

TABLEAU 2.2 – Tableau des besoins fonctionnels

ID	Description	Priorité	Acteur concerné
BF-01	S'inscrire avec email/mot de passe	Must have	Développeur
BF-02	S'authentifier via OAuth GitHub	Must have	Développeur
BF-03	Émettre et renouveler des jetons JWT	Must have	Système (NestJS)
BF-04	Créer un projet depuis une URL GitHub	Must have	Développeur
BF-05	Consulter la liste des projets et leurs statuts	Must have	Développeur
BF-06	Déclencher manuellement un déploiement	Must have	Développeur

Suite à la page suivante

ID	Description	Priorité	Acteur concerné
BF-07	Détecter automatiquement le framework applicatif	Must have	Système (Builder Go)
BF-08	Construire l'image conteneur via Buildkit/Railpack	Must have	Système (Argo Workflows)
BF-09	Déployer l'application via chart Helm	Must have	Système (Argo Workflows)
BF-10	Consulter les journaux de build en temps réel (SSE)	Must have	Développeur
BF-11	Gérer les variables d'environnement (CRUD)	Must have	Développeur
BF-12	Stocker les secrets dans Hashi-Corp Vault	Must have	Système (NestJS/ESO)
BF-13	Configurer le déploiement automatique sur push GitHub	Should have	Développeur
BF-14	Consulter l'historique des déploiements	Should have	Développeur
BF-15	Effectuer un retour arrière vers une version précédente	Should have	Développeur
BF-16	Supprimer un projet avec déprovisionnement Kubernetes	Must have	Développeur
BF-17	Gérer les membres d'une équipe projet	Should have	Développeur
BF-18	Administrer les tenants et ressources (interface admin)	Must have	Administrateur
BF-19	Visualiser les workflows Argo en cours	Should have	Administrateur
BF-20	Configurer un domaine personnalisé	Could have	Développeur

2.1.3 Besoins non fonctionnels

Les besoins non fonctionnels définissent les contraintes de qualité auxquelles le système doit se conformer, indépendamment des fonctionnalités qu’il offre. Ils couvrent des attributs tels que la performance, la disponibilité, la sécurité et la scalabilité, et constituent souvent les critères les plus déterminants pour l’expérience utilisateur réelle et la viabilité opérationnelle de la plateforme [3]. Dans le contexte d’une plateforme PaaS multi-tenant comme Hostifer, ces exigences sont particulièrement critiques, car une dégradation dans l’un de ces domaines peut simultanément affecter l’ensemble des tenants hébergés sur l’infrastructure partagée.

Performance. Le temps de déploiement de bout en bout, mesuré depuis la soumission du workflow Argo jusqu’au passage de l’application à l’état **LIVE**, ne doit pas excéder cinq minutes pour les frameworks courants (Express, NestJS, Next.js, Flask) sur un dépôt de taille standard. La latence de streaming des journaux de build via SSE doit rester inférieure à deux secondes entre l’émission d’un message par le pod de construction et son affichage dans le navigateur de l’utilisateur.

Disponibilité. La plateforme cible un SLA (*Service Level Agreement*) de disponibilité de 99,5% pour les plans Standard, et de 99,9% pour les plans Enterprise, ce qui correspond à une indisponibilité maximale tolérée respectivement de 43,8 heures et 8,76 heures par an. Ces objectifs imposent une architecture résiliente aux défaillances de composants individuels, notamment via la redondance des pods applicatifs et la gestion des interruptions planifiées.

Sécurité. L’isolation réseau entre tenants doit être garantie par des *NetworkPolicies* Kubernetes bloquant tout trafic inter-namespace non autorisé [6]. Les secrets applicatifs ne doivent jamais transiter en clair : leur cycle de vie complet (création, stockage, injection dans les pods) est pris en charge par HashiCorp Vault et l’opérateur External Secrets. Les communications entre l’utilisateur et la plateforme doivent être intégralement chiffrées via TLS, avec émission et renouvellement automatiques des certificats assurés par cert-manager et Let’s Encrypt. L’accès aux ressources Kubernetes est contrôlé par des politiques RBAC (*Role-Based Access Control*) granulaires, limitant strictement les permissions de chaque composant.

Scalabilité et autoscaling. La plateforme doit supporter une montée en charge dynamique à trois niveaux distincts [7], correspondant aux trois couches de l’architecture Kubernetes.

Au niveau des pods, le *Horizontal Pod Autoscaler* (HPA) ajuste automatiquement le nombre de réplicas d’un déploiement en fonction de métriques observées telles que l’utilisation CPU ou mémoire, ou de métriques applicatives personnalisées exposées via

Prometheus Adapter [8]. Le HPA est configuré avec des seuils de tolérance adaptés afin d’éviter les phénomènes d’oscillation (*flapping*) dus à des variations métrique transitoires. En complément, le *Vertical Pod Autoscaler* (VPA) opère sur la dimension verticale en ajustant les champs `requests` et `limits` des conteneurs sur la base de l’historique de consommation réelle [9]. En mode `recommendation`, il fournit des suggestions sans perturber les pods en cours d’exécution ; en mode `auto`, il peut procéder à l’éviction et au redémarrage des pods pour appliquer les nouvelles valeurs. Afin d’éviter tout conflit avec le HPA sur les mêmes métriques, le VPA est configuré pour opérer sur des métriques distinctes lorsque les deux mécanismes coexistent [10].

Au niveau du cluster, des *node pools* classés par profil de ressources (small, medium, large) permettent d’affecter les workloads aux nœuds les mieux dimensionnés selon le plan tarifaire du tenant. Le *Cluster Autoscaler* surveille en permanence l’état des pods en attente de scheduling (*Pending*) et des nœuds sous-utilisés, ajoutant de nouveaux nœuds lorsque la demande dépasse la capacité disponible et en retirant lorsque des nœuds restent inactifs sur une période prolongée [7].

Les *ResourceQuotas* et les *LimitRanges* définis par namespace délimitent les bornes minimales et maximales de consommation de ressources par tenant. Ces contraintes co-opèrent avec le HPA et le VPA en empêchant les recommandations d’autoscaling de dépasser les quotas alloués par plan, garantissant ainsi la préservation de l’isolation multi-tenant même en période de forte charge [11].

Maintenabilité. L’architecture modulaire du backend NestJS (un module par domaine fonctionnel), l’utilisation de charts Helm paramétrables pour le provisionnement des tenants, et la séparation stricte des composants (frontend, backend, builder, infrastructure) doivent permettre la mise à jour et le débogage indépendants de chaque couche du système sans affecter les tenants en production.

Le tableau 2.3 récapitule l’ensemble des besoins non fonctionnels avec leurs critères de satisfaction mesurables.

TABLEAU 2.3 – Tableau des besoins non fonctionnels

ID	Catégorie	Description	Critère de satisfaction
BNF-01	Performance	Temps de déploiement de bout en bout	≤ 5 min pour frameworks standard
BNF-02	Performance	Latence de streaming des logs SSE	≤ 2 s entre émission pod et affichage navigateur

ID	Catégorie	Description	Critère de satisfaction
BNF-03	Disponibilité	SLA plan Standard	$\geq 99,5\%$ (43,8 h d'indisponibilité/an max.)
BNF-04	Disponibilité	SLA plan Enterprise	$\geq 99,9\%$ (8,76 h d'indisponibilité/an max.)
BNF-05	Sécurité	Isolation réseau inter-tenant	Zéro trafic inter-namespace validé par NetworkPolicy
BNF-06	Sécurité	Chiffrement des secrets	Secrets stockés dans Vault KV v2, jamais en clair
BNF-07	Sécurité	Chiffrement des communications	TLS obligatoire, certificats émis par cert-manager
BNF-08	Sécurité	Contrôle d'accès	RBAC Kubernetes par namespace et par composant
BNF-09	Scalabilité	Autoscaling horizontal des pods (HPA)	Augmentation du nombre de réplicas en ≤ 30 s sous charge CPU $> 70\%$
BNF-10	Scalabilité	Autoscaling vertical des pods (VPA)	Recommandations requests/limits appliquées sans interruption en mode recommendation
BNF-11	Scalabilité	Autoscaling du cluster (Cluster Autoscaler)	Ajout/suppression automatique de nœuds en ≤ 3 min selon utilisation des ressources

ID	Catégorie	Description	Critère de satisfaction
BNF-12	Scalabilité	Isolation des quotas multi-tenant	ResourceQuota empêche tout dépassement du plan tarifaire, même en cas de scaling
BNF-13	Maintenabilité	Modularité du backend	Mise à jour d'un module NestJS sans impact sur les autres
BNF-14	Maintenabilité	Paramétrabilité de l'infrastructure	Provisionnement complet d'un nouveau tenant via chart Helm sans modification manuelle

2.2 Modélisation UML des cas d'utilisation

2.2.1 Cas d'utilisation détaillés

Les cas d'utilisation suivants sont décrits selon le format textuel standard préconisé pour les mémoires de fin d'études en informatique. Chaque fiche précise le nom du cas d'utilisation, l'acteur principal, les préconditions requises, le scénario nominal (flux de succès), les scénarios alternatifs et d'exception, ainsi que les postconditions observables à l'issue de l'interaction.

2.2.2 Diagramme de cas d'utilisation global

2.2.3 Cas d'utilisation détaillés

2.2.3.1 CU01 : S'authentifier

2.2.3.2 CU02 : Créer un projet depuis une URL GitHub

2.2.3.3 CU03 : Configurer et déclencher un déploiement

2.2.3.4 CU04 : Consulter les logs de build en temps réel

2.2.3.5 CU05 : Gérer les variables d'environnement

2.2.3.6 CU06 : Configurer le CI/CD

2.2.3.7 CU07 : Gérer les membres d'une équipe

2.2.3.8 CU08 : Supprimer un projet

2.2.3.9 CU09 : Administrer la plateforme

TABLEAU 2.4 – Tableau récapitulatif des cas d'utilisation (ID, nom, acteur principal, priorité)

2.3 Modélisation des classes et données

2.3.1 Diagramme de classes

2.3.2 Modèle Entité-Association

2.3.3 Description des entités principales

La modélisation des entités constitue le socle de la couche de persistance de la plateforme. Chaque entité est implémentée sous forme de modèle Prisma ORM et correspond à une table dans la base de données PostgreSQL. Les tableaux suivants décrivent, pour chaque entité principale, l'ensemble des attributs avec leurs types de données, leurs contraintes d'intégrité, et leurs relations avec les autres entités du schéma.

TABLEAU 2.5 – Description de l'entité User

Attribut	Type	Contraintes	Description
id	String	PK, unique, cuid()	Identifiant unique de l'utilisateur généré automatiquement
email	String	unique, non null	Adresse email de l'utilisateur, utilisée pour l'authentification
password	String?	nullable	Mot de passe haché (bcrypt); null si authentification OAuth uniquement
name	String?	nullable	Nom complet de l'utilisateur
avatarUrl	String?	nullable	URL de l'avatar (fourni par GitHub OAuth)
role	Enum	non null, défaut USER	Rôle de l'utilisateur : USER ou ADMIN
githubId	String?	unique, nullable	Identifiant GitHub de l'utilisateur; null si inscription email
refreshToken	String?	nullable	Dernier jeton de rafraîchissement JWT émis, haché en base
createdAt	DateTime	non null, défaut now()	Date et heure de création du compte
updatedAt	DateTime	non null, @updatedAt	Date et heure de la dernière mise à jour
Relations			

Suite à la page suivante

Attribut	Type	Contraintes	Description
projects	Project[]	—	Projets dont l'utilisateur est propriétaire
memberships	ProjectMember[]	—	Appartenances à des projets en tant que membre d'équipe
installations	GitHubInstallation[]	—	Installations GitHub OAuth associées au compte
activityLogs	ActivityLog[]	—	Journal des actions effectuées par l'utilisateur
notifications	Notification[]	—	Notifications reçues par l'utilisateur

TABLEAU 2.6 – Description de l'entité Project

Attribut	Type	Contraintes	Description
id	String	PK, unique, cuid()	Identifiant unique du projet
name	String	non null	Nom du projet, utilisé comme sous-domaine de l'URL publique
slug	String	unique, non null	Version normalisée du nom, utilisée dans les ressources Kubernetes et les URLs
ownerId	String	FK → User.id, non null	Référence à l'utilisateur propriétaire du projet

Suite à la page suivante

Attribut	Type	Contraintes	Description
repoUrl	String	non null	URL du dépôt GitHub source
repoName	String	non null	Nom du dépôt GitHub (format <code>owner/repo</code>)
branch	String	non null, défaut "main"	Branche de déploiement par défaut
framework	String?	nullable	Framework détecté automatiquement lors du dernier build
planId	String	FK → Plan.id, non null	Plan tarifaire associé au projet (détermine les quotas de ressources)
autoDeploy	Boolean	non null, défaut false	Active le déploiement automatique sur événement <code>push</code> GitHub
webhookSecret	String?	nullable	Secret HMAC pour la validation des événements webhook GitHub
installationId	String	FK → GitHubInstallation.id, non null	Installation GitHub permettant l'accès au dépôt
namespace	String?	unique, nullable	Namespace Kubernetes alloué au projet lors du premier déploiement
publicUrl	String?	nullable	URL publique de l'application déployée

Suite à la page suivante

Attribut	Type	Contraintes	Description
createdAt	DateTime	non null, défaut now()	Date et heure de création du projet
updatedAt	DateTime	non null, @updatedAt	Date et heure de la dernière mise à jour
Relations			
owner	User	FK → User	Utilisateur propriétaire du projet
plan	Plan	FK → Plan	Plan tarifaire définissant les quotas de ressources
deployments	Deployment []	—	Historique de tous les déploiements du projet
envVars	EnvVar []	—	Variables d'environnement associées au projet
members	ProjectMember []	—	Membres de l'équipe ayant accès au projet
domains	Domain []	—	Domaines personnalisés associés au projet
activityLogs	ActivityLog []	—	Journal des actions effectuées sur le projet

TABLEAU 2.7 – Description de l'entité Deployment

Attribut	Type	Contraintes	Description
id	String	PK, unique, cuid()	Identifiant unique du déploiement

Suite à la page suivante

Attribut	Type	Contraintes	Description
projectId	String	FK → Project.id, non null	Référence au projet auquel appartient ce déploiement
status	Enum	non null, défaut QUEUED	État courant du dé- ploiement (voir cycle de vie ci-dessous)
commitSha	String?	nullable	Hash du commit Git déployé
commitMessage	String?	nullable	Message du commit Git associé au déploie- ment
branch	String	non null	Branche Git source du déploiement
imageTag	String?	nullable	Tag de l'image OCI construite et poussée vers le registre interne
argoWorkflowName	String?	nullable	Nom du workflow Argo Workflows soumis pour ce dé- ploiement
buildDuration	Int?	nullable	Durée de l'étape Build en secondes
totalDuration	Int?	nullable	Durée totale du dé- ploiement (Build + Provision + DNS) en secondes
triggeredBy	Enum	non null	Origine du déclenche- ment : MANUAL ou WEBHOOK

Suite à la page suivante

Attribut	Type	Contraintes	Description
createdAt	DateTime	non null, défaut now()	Date et heure de création de l'entrée de déploiement
updatedAt	DateTime	non null, @updatedAt	Date et heure de la dernière mise à jour de l'état
finishedAt	DateTime?	nullable	Date et heure de fin du déploiement (succès ou échec)
Relations			
project	Project	FK → Project	Projet parent du déploiement
steps	DeploymentStep[]		Étapes détaillées du pipeline (Build, Provision, DNS)
Valeurs de l'énumération DeploymentStatus			
QUEUED → BUILDING → PROVISIONING → CONFIGURING_DNS → LIVE Transitions d'échec possibles depuis tout état actif : → FAILED Annulation possible depuis tout état actif : → CANCELLED			

TABLEAU 2.8 – Description de l'entité User (attributs, types, contraintes)**TABLEAU 2.9** – Description de l'entité Project**TABLEAU 2.10** – Description de l'entité Deployment

2.4 Modélisation comportementale

2.4.1 Diagrammes de séquence

2.4.2 Diagrammes d'activité

2.4.3 Diagramme d'états

2.5 Architecture technique globale

2.5.1 Vue d'ensemble de l'architecture

2.5.2 Architecture multi-tenant

TABLEAU 2.11 – Ressources allouées par profil d'allocation (CPU request/limit, RAM request/limit, stockage, pods max). Les métriques d'autoscaling supportées et les comportements HPA/VPA autorisés par profil sont indiqués dans la grille

2.5.3 Architecture du pipeline de build

2.5.4 Architecture de collecte des logs

2.5.5 Architecture de gestion des secrets

2.6 Choix technologiques

2.6.1 Justification des technologies retenues

TABLEAU 2.12 – Tableau des choix technologiques (composant, technologie retenue, alternatives considérées, justification du choix)

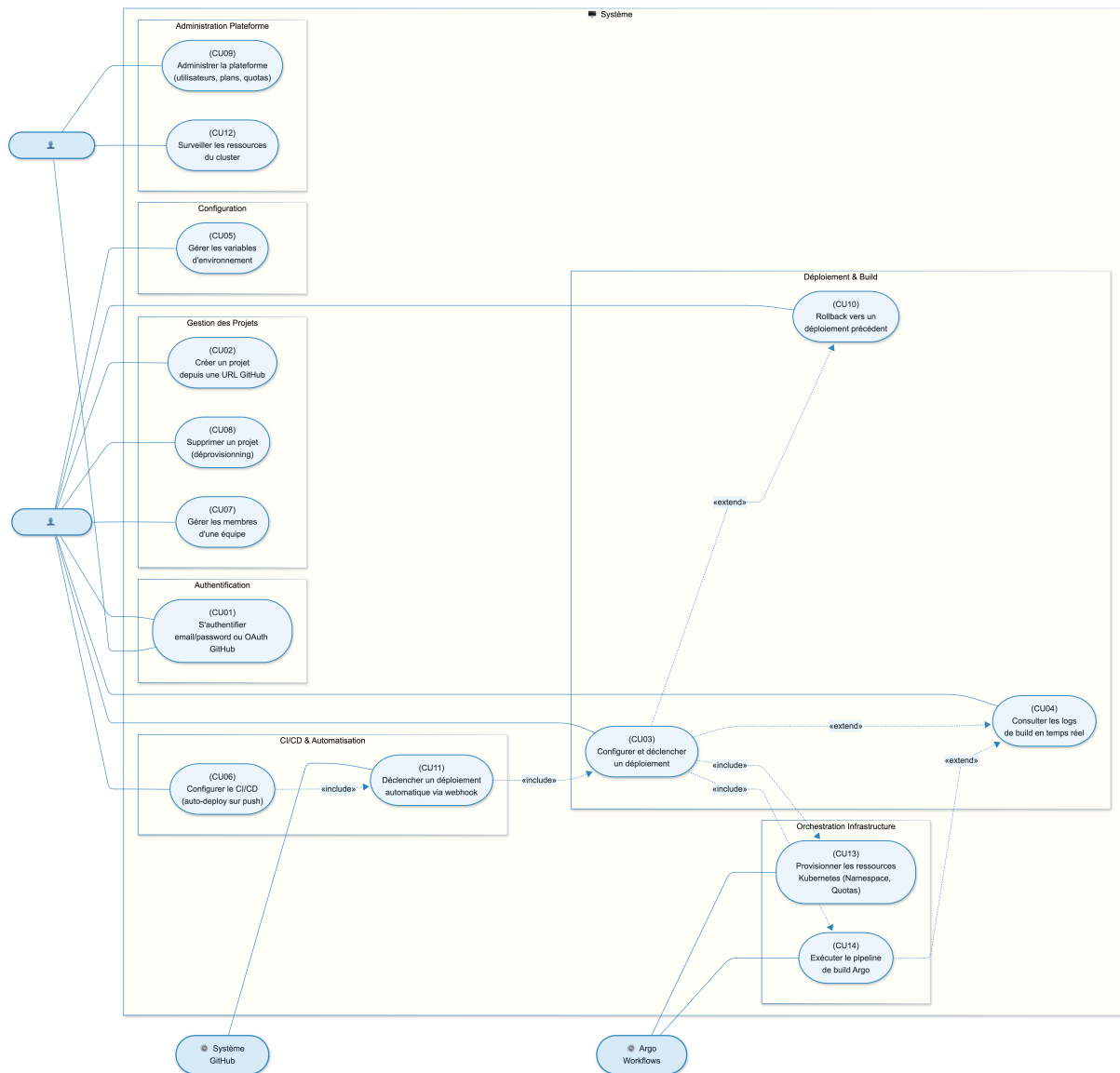


FIGURE 2.1 – Diagramme de cas d'utilisation global montrant tous les acteurs et leurs interactions avec le système (authentification, déploiement, gestion des projets, administration, CI/CD, logs)

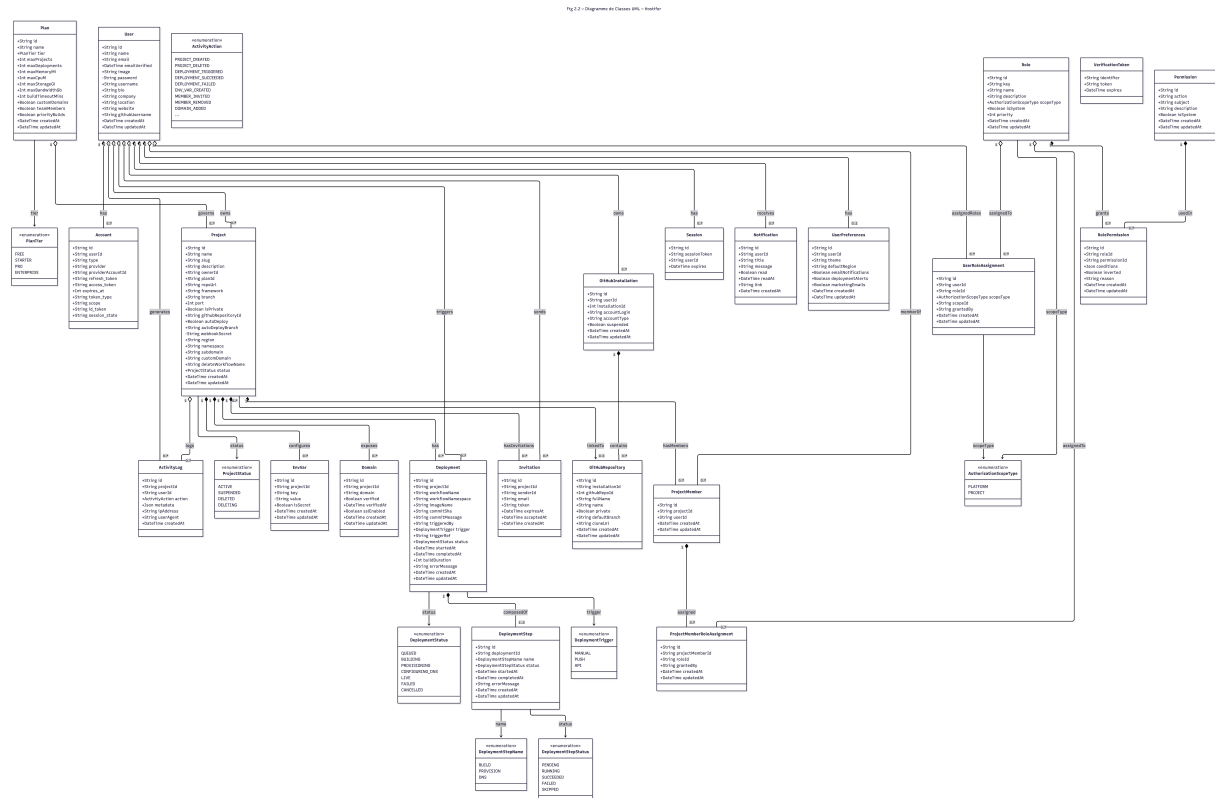


FIGURE 2.2 – Diagramme de classes UML complet (User, Project, Deployment, DeploymentStep, EnvVar, Plan, GitHubInstallation, GitHubRepository, ActivityLog, Notification, Domain)



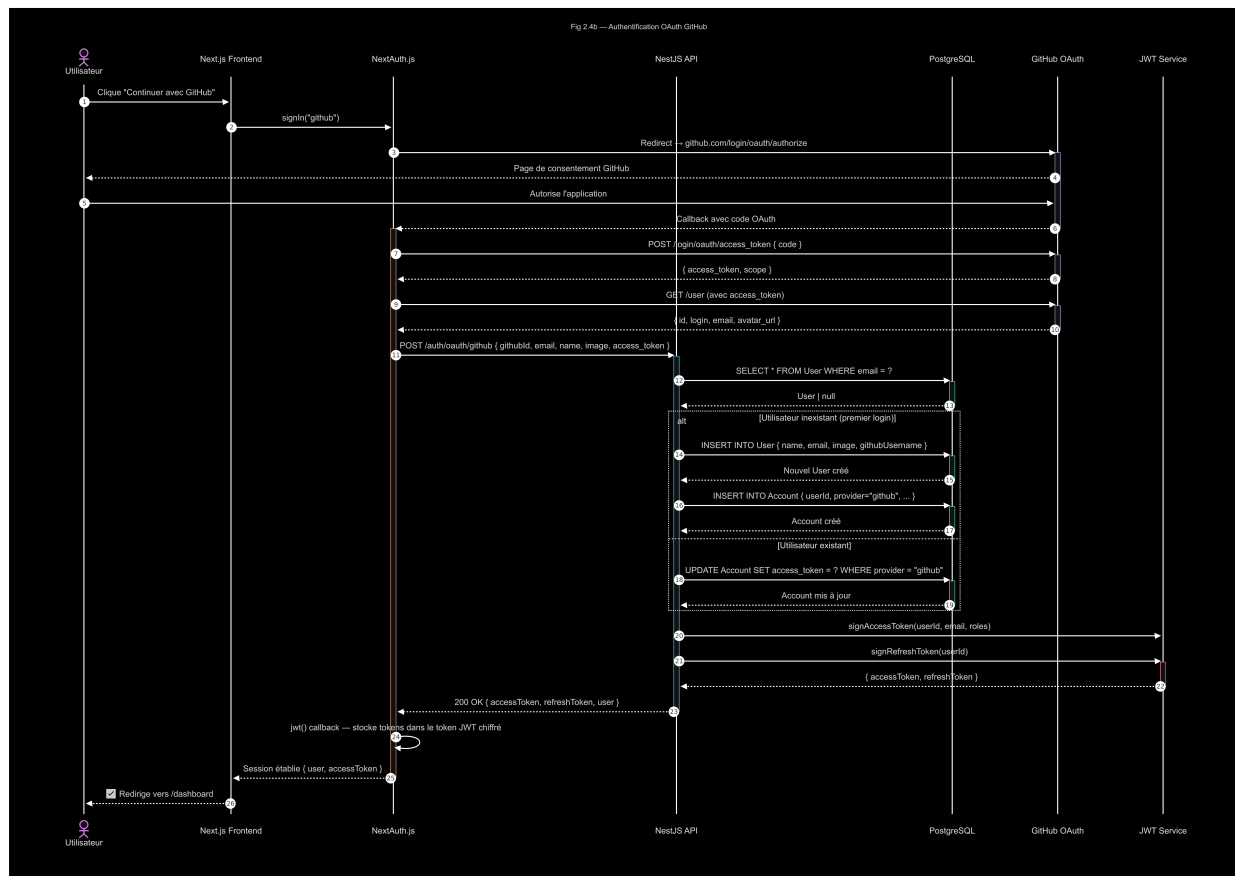


FIGURE 2.4 – Diagramme de séquence : Authentification (NextJS → NestJS → PostgreSQL → JWT)

FIGURE 2.5 – Diagramme de séquence : Déclenchement d'un déploiement (Frontend → NestJS → Argo Workflows → Build Job → Registry → Helm → DNS)

FIGURE 2.6 – Diagramme de séquence : Streaming des logs en temps réel (Builder Pod → Promtail → Loki → NestJS SSE → Browser)

FIGURE 2.7 – Diagramme de séquence : Webhook CI/CD GitHub Push (GitHub → NestJS → DeploymentsService → Argo Workflows)

FIGURE 2.8 – Diagramme d'activité du pipeline de déploiement complet (Build → Provision → DNS → Live)

FIGURE 2.9 – Diagramme d'activité de la détection de framework et génération du plan Rail-pack

FIGURE 2.10 – Diagramme d'états d'un Deployment (QUEUED → BUILDING → PROVISIONING → CONFIGURING_DNS → LIVE / FAILED / CANCELLED)

FIGURE 2.11 – Schéma d’architecture générale de Hostifer : couche client (NextJS), couche API (NestJS), couche orchestration (Argo Workflows), couche infrastructure (Kubernetes, Buildkit, Registry, Loki, Vault), couche réseau (Traefik, Cloudflare, cert-manager)

FIGURE 2.12 – Schéma d’isolation multi-tenant : Namespace par projet, NetworkPolicy bloquant le trafic inter-tenant, ressources quotas par plan

FIGURE 2.13 – Schéma du pipeline de build (Job Kubernetes Go binary → Railpack CLI → Buildkitd Deployment → Registry interne)

FIGURE 2.14 – Schéma de la chaîne de collecte et streaming des logs (Pod stdout → Promtail DaemonSet → Loki → SSE → Browser)

FIGURE 2.15 – Schéma de gestion des secrets avec HashiCorp Vault et External Secrets Operator

Chapitre 3

Réalisation et Implémentation

3.1 Environnement de développement et de déploiement

3.1.1 Infrastructure matérielle

L'infrastructure de déploiement de Hostifer repose entièrement sur le service *Container Service for Kubernetes* (ACK) d'Alibaba Cloud, un service Kubernetes managé certifié conforme au programme *Kubernetes Conformance* du CNCF [12]. Le choix d'ACK est motivé par son intégration native avec l'ensemble des briques réseau, stockage et sécurité d'Alibaba Cloud, ainsi que par la prise en charge managée du plan de contrôle, qui délègue à Alibaba Cloud la responsabilité opérationnelle des composants critiques (*kube-apiserver*, *kube-controller-manager*, *kube-scheduler*, *etcd*) [13].

L'architecture de déploiement s'articule autour d'un réseau privé virtuel (*Virtual Private Cloud*, VPC) unique constituant le périmètre d'isolation réseau de l'ensemble des ressources de la plateforme [14]. Au sein de ce VPC, un *vSwitch* dédié héberge les nœuds du cluster en leur attribuant des adresses IP privées, conformément aux bonnes pratiques de planification réseau ACK préconisant la séparation entre le sous-réseau des nœuds et les blocs CIDR des pods et des services [15].

Cluster ACK et node pools

Le cluster ACK est configuré en mode *managed Pro*, dans lequel Alibaba Cloud héberge et maintient intégralement le plan de contrôle dans son infrastructure méta-cluster (*Kube-on-Kube*), garantissant un SLA de disponibilité du plan de contrôle de 99,95% en configuration multi-zone [13]. Le plan de données est composé de trois node pools distincts, dimensionnés selon les profils de charge de la plateforme :

- **Node pool du plan de contrôle ACK (3 nœuds)** : ce pool est réservé aux composants internes du plan de contrôle gérés par ACK (notamment *CCM*, *etcd* et les services système Kubernetes associés), et n'héberge pas les workloads applicatifs de la plateforme.
- **Node pool des services principaux (3 nœuds)** : trois instances ECS configurées avec 4 vCPU et 12 Go de RAM, dédiées aux fonctionnalités cœur de la plateforme (backend NestJS, Argo Workflows, Loki, Vault, cert-manager, Traefik, registre OCI). Ce dimensionnement offre un compromis entre capacité mémoire et coût pour des composants d'orchestration et d'observabilité exécutés en continu.
- **Node pool des tenants (3 nœuds, extensible)** : trois instances ECS configurées avec 4 vCPU et 16 Go de RAM, dédiées à l'exécution des workloads applicatifs des tenants et des jobs de construction Buildkit. Ce node pool est géré par le Cluster Autoscaler d'ACK, qui ajoute ou supprime des nœuds automatiquement en fonction de l'état des pods *Pending* et du taux d'utilisation des ressources, permettant une élasticité horizontale sans intervention manuelle [12].

Architecture réseau

La topologie réseau de la plateforme repose sur deux *Server Load Balancers* (SLB) [16] et trois *Elastic IP Addresses* (EIP) [17], dont les rôles sont strictement distincts :

SLB d'entrée applicative (ports 80 et 443) : ce premier équilibreur de charge reçoit l'ensemble du trafic HTTP et HTTPS provenant des utilisateurs finaux et des webhooks GitHub. Il distribue le trafic entrant vers le pod Traefik déployé sur les nœuds du cluster, qui assure ensuite le routage vers les services des tenants selon les règles d'IngressRoute. Une EIP entrante dédiée lui est associée, constituant le point d'entrée public unique de la plateforme.

SLB d'accès à l'API Kubernetes (port 6443) : ce second équilibreur de charge expose le serveur API Kubernetes (*kube-apiserver*) pour les connexions *kubectl* et les appels d'administration depuis les outils de déploiement (Helm, Argo Workflows CLI). Une EIP entrante dédiée lui est associée, permettant l'accès sécurisé au plan de contrôle depuis les environnements de développement et de CI/CD.

NAT Gateway et EIP sortante : les nœuds de travail ne disposent d'aucune adresse IP publique directe. Leurs connexions sortantes (clonage de dépôts Git depuis GitHub, téléchargement d'images de base depuis des registres publics) sont acheminées via une passerelle NAT Internet (*Internet NAT Gateway*) à laquelle une troisième EIP sortante est associée [14]. Ce mécanisme de SNAT centralise et masque les origines des requêtes sortantes, réduisant la surface d'exposition publique du cluster.

Infrastructure de stockage

La plateforme mobilise trois types de stockage complémentaires, sélectionnés selon les caractéristiques d'accès et de persistance requises par chaque composant [18] :

NAS (*Network Attached Storage*) : le stockage NAS est utilisé pour les *Persistent-VolumeClaims* (PVC) nécessitant un accès concurrent depuis plusieurs pods, notamment pour le registre OCI interne (stockage des couches d'images) et pour la persistance des données Loki. Le protocole NFS sous-jacent permet le montage simultané depuis plusieurs nœuds (*ReadWriteMany*), ce qui est incompatible avec les disques EBS par nature mono-nœud [19].

Cloud Disks EBS (ESSD) : les volumes EBS de type ESSD (*Enhanced SSD*) sont utilisés pour les workloads nécessitant des accès disque à faible latence et à haute performance I/O, notamment les jobs de construction Buildkitd pour lesquels le cache des couches de build est stocké localement sur des volumes bloc dédiés. Chaque volume EBS est attaché à un nœud unique (*ReadWriteOnce*), ce qui les rend adaptés aux charges à fort débit séquentiel ou aléatoire [18].

Object Storage Service (OSS) : le stockage objet OSS est utilisé pour deux cas d'usage distincts. D'une part, le stockage occasionnel de fichiers artefacts générés par la plateforme (plans de build exportés, archives de logs). D'autre part, le stockage dit *cold* pour les journaux et données d'observabilité archivés à long terme, pour lesquels les contraintes de latence d'accès sont relâchées mais le coût de stockage par gigaoctet doit être minimisé [19].

Le tableau 3.1 récapitule les caractéristiques de l'environnement de déploiement.

TABEAU 3.1 – Caractéristiques de l'environnement de déploiement

Composant	Paramètre	Valeur
Cluster Kubernetes	Service	Alibaba Cloud ACK (<i>Managed Pro</i>)
	Distribution	Kubernetes 1.31+ (ACK patch x.y.z-aliyun.n)
	Runtime	containerd
Node pool contrôle	Nœuds	3 instances ECS
	CPU	4 vCPU / nœud
	RAM	16 Go / nœud

Suite à la page suivante

Composant	Paramètre	Valeur
	OS	Alibaba Cloud Linux
Node pool travail	Nœuds	3 instances ECS (extensible)
	CPU	4 vCPU / nœud
	RAM	8 Go / nœud
	OS	Alibaba Cloud Linux
	Autoscaling	Cluster Autoscaler ACK (ajout/suppression automatique de nœuds)
Réseau	VPC	1 VPC isolé, 1 vSwitch nœuds
	SLB applicatif	1 SLB — ports 80/443 (trafic HTTP/HTTPS entrant)
	SLB API server	1 SLB — port 6443 (accès <code>kubectl</code>)
	EIP entrante (app)	1 EIP associée au SLB ports 80/443
	EIP entrante (API)	1 EIP associée au SLB port 6443
Stockage	EIP sortante	1 EIP associée au NAT Gateway (SNAT worker nodes)
	NAS (NFS)	PVC multi-nœuds (<i>ReadWriteMany</i>) — registre OCI, Loki
	EBS (ESSD)	Volumes bloc haute performance (<i>ReadWriteOnce</i>) — cache Buildkitd
	OSS	Stockage objet — artefacts occasionnels et archivage <i>cold</i>

3.1.2 Outils de développement

TABLEAU 3.2 – Environnement de développement

Catégorie	Outil / Technologie	Rôle
IDE	Visual Studio Code	Éditeur principal pour le développement frontend, backend et infrastructure
Runtimes	Node.js 24 LTS	Environnement d'exécution JavaScript pour le frontend Next.js et le backend NestJS
	Go (dernière version LTS)	Environnement d'exécution du binaire builder
Frameworks	Next.js	Framework React full-stack pour le frontend de la plateforme
	NestJS	Framework Node.js modulaire pour le backend API REST et SSE
Base de données	PostgreSQL	Système de gestion de base de données relationnelle principal de la plateforme
Gestionnaires de paquets	npm	Gestion des dépendances Node.js (frontend Next.js et backend NestJS)
	Go Modules (go mod)	Gestion des dépendances du binaire builder Go
	Helm	Gestion et déploiement des charts Kubernetes de la plateforme
Outils CLI	kubectl	Administration et inspection des ressources du cluster Kubernetes
	helm	Déploiement, mise à jour et test des charts Helm en local et en production

Suite à la page suivante

Catégorie	Outil / Technologie	Rôle
	argo	Soumission, suivi et débogage des workflows Argo Workflows
	crictl	Inspection bas niveau des conteneurs et images via l'interface CRI
	docker	Construction et test local des images conteneurs avant push vers le registre
	git	Gestion du code source et interaction avec les dépôts GitHub
	vault	Interaction avec HashiCorp Vault pour la gestion et le test des secrets
	railpack	Test local de la détection de framework et de la génération des plans de build
Chaîne CI/CD	GitHub Actions	Intégration continue : lint, build, tests et publication des images OCI sur GHCR
	ArgoCD	Livraison continue : synchronisation GitOps des manifests Kubernetes vers le cluster ACK

3.1.3 Organisation des dépôts GitHub

Le code source de Hostifer est organisé en plusieurs dépôts GitHub distincts, chacun correspondant à un composant logique de la plateforme. Cette séparation permet une gestion indépendante des cycles de release, des pipelines CI/CD et des droits d'accès pour chaque composant. Les artefacts produits par chaque dépôt — images Docker et charts Helm — sont publiés sur le registre *GitHub Container Registry* (GHCR) sous le namespace `ghcr.io/hostifer`, conformément au standard OCI (*Open Container Initiative*) [20].

Dépôt frontend (Next.js). Ce dépôt contient le code source de l'interface utilisateur

de la plateforme, développée avec Next.js. Le pipeline GitHub Actions associé construit et publie l'image Docker du frontend sur GHCR à l'adresse `ghcr.io/hostifer/frontend:<tag>`. Cette image est ensuite déployée sur le cluster ACK via ArgoCD lors de chaque release.

Dépôt backend (NestJS). Ce dépôt contient le code source de l'API backend, développée avec NestJS. Son pipeline CI/CD produit deux images Docker distinctes publiées sur GHCR : l'image principale du backend à l'adresse

`ghcr.io/hostifer/backend:<tag>`, et une image de migration de schéma de base de données à l'adresse `ghcr.io/hostifer/migrate:<tag>`. Cette dernière est exécutée en tant que Job Kubernetes avant chaque mise à jour du backend afin d'appliquer les migrations Prisma sans interruption de service.

Dépôt helm (chart principal). Ce dépôt contient le chart Helm de déploiement des composants principaux de la plateforme (frontend, backend, ingress, secrets). Le chart est publié sous forme de package OCI sur GHCR à l'adresse `oci://ghcr.io/hostifer/charts/helm` et est consommé par ArgoCD pour la synchronisation GitOps de l'état du cluster.

Dépôt tenant (chart tenant). Ce dépôt contient le chart Helm dédié au provisionnement des tenants. À chaque déploiement d'application utilisateur, Argo Workflows invoque ce chart publié à l'adresse `oci://ghcr.io/hostifer/charts/tenant` pour créer l'ensemble des ressources Kubernetes du namespace tenant : *Deployment*, *Service*, *IngressRoute*, *NetworkPolicy*, *ResourceQuota*, *HorizontalPodAutoscaler* et *ExternalSecret*.

Dépôt build-job (Go). Ce dépôt contient le code source du binaire Go exécuté en tant que Job Kubernetes lors de l'étape *Build* du pipeline Argo Workflows. Il orchestre le clonage du dépôt Git, l'invocation de Railpack CLI pour la détection du framework et la génération du plan de build, puis la construction et le push de l'image OCI de l'application via Buildkitd. L'image du builder est publiée sur GHCR à l'adresse `ghcr.io/hostifer/build-job:<tag>`.

Dépôt create-dns-job (Go). Ce dépôt contient le code source du Job Go responsable de la création des enregistrements DNS pour le sous-domaine de l'application déployée, lors de l'étape *DNS* du pipeline Argo Workflows. Son image Docker est publiée à l'adresse `ghcr.io/hostifer/create-dns:<tag>`.

Dépôt delete-dns-job (Go). Ce dépôt contient le code source du Job Go déclenché lors de la suppression d'un projet (CU08), chargé de supprimer les enregistrements DNS associés au sous-domaine de l'application. Son image Docker est publiée à l'adresse `ghcr.io/hostifer/delete-dns-job:<tag>`.

Dépôt gitops (ArgoCD). Ce dépôt ne produit aucun artefact compilé. Il contient exclusivement les manifests Kubernetes et les fichiers de valeurs Helm (`values.yaml`) décrivant l'état cible du cluster. ArgoCD surveille ce dépôt en continu et synchronise l'état réel du cluster ACK vers l'état déclaré dans le dépôt, selon le paradigme GitOps [21].

Dépôt documentation (Mintlify). Ce dépôt contient la documentation technique et utilisateur de la plateforme, générée avec Mintlify. Il ne produit aucun artefact OCI et est déployé indépendamment via le service d'hébergement Mintlify.

Le tableau 3.3 récapitule l'ensemble des dépôts, leurs technologies et les artefacts publiés.

TABLEAU 3.3 – Organisation des dépôts GitHub et artefacts GHCR

Dépôt	Langage	Artefacts publiés (GHCR)
frontend	TypeScript	ghcr.io/hostifer/frontend:<tag>
backend	TypeScript	ghcr.io/hostifer/backend:<tag> ghcr.io/hostifer/migrate:<tag>
helm	Helm	oci://ghcr.io/hostifer/charts/helm
tenant	Helm	oci://ghcr.io/hostifer/charts/tenant
build-job	Go	ghcr.io/hostifer/build-job:<tag>
create-dns-job	Go	ghcr.io/hostifer/create-dns:<tag>
delete-dns-job	Go	ghcr.io/hostifer/delete-dns-job:<tag>
gitops	YAML	— (aucun artefact, manifests ArgoCD uniquement)
documentation	MDX	— (aucun artefact, hébergement Mintlify)

FIGURE 3.1 – Schéma d'organisation des dépôts GitHub et registres OCI

3.2 Implémentation du backend NestJS

3.2.1 Structure modulaire de l'application

FIGURE 3.2 – Arborescence du projet NestJS avec description de chaque module (Auth, Users, Projects, Deployments, Logs, EnvVars, GitHub, Plans, Infrastructure)

3.2.2 Module d'authentification

Le module d'authentification constitue l'une des couches les plus critiques de la plateforme. Il est implémenté dans le module NestJS `auth/` et repose sur la bibliothèque Passport.js, intégrée à NestJS via le package `@nestjs/passport`, qui expose un mécanisme de stratégies d'authentification interchangeable et extensibles [22]. Trois mécanismes d'authentification distincts sont implémentés : l'authentification par identifiant et mot de passe avec émission de jetons JWT, l'authentification déléguée via OAuth 2.0 GitHub, et l'authentification à deux facteurs (2FA) basée sur TOTP (*Time-based One-Time Password*).

Authentification JWT et stratégie Passport

L'authentification principale repose sur la stratégie `passport-jwt`, configurée dans `jwt.strategy.ts`. Lors d'une connexion par identifiant et mot de passe, le service `auth.service.ts` vérifie les identifiants soumis contre l'enregistrement `User` en base de données PostgreSQL via Prisma, en comparant le mot de passe soumis au hachage bcrypt stocké. En cas de succès, il émet deux jetons distincts via le service `@nestjs/jwt` [23] :

- Un **jeton d'accès** (*access token*) JWT de courte durée de vie, signé avec un secret `JWT_ACCESS_SECRET` dédié. Son payload contient uniquement l'identifiant et l'email de l'utilisateur, conformément au principe de minimisation des données dans le payload JWT [24]. Ce jeton est transmis par le client dans l'en-tête `Authorization: Bearer <token>` à chaque requête vers les endpoints protégés.
- Un **jeton de rafraîchissement** (*refresh token*) de longue durée de vie, signé avec un secret `JWT_REFRESH_SECRET` distinct. Avant persistance, ce jeton est haché en base de données dans le champ `refreshToken` de l'entité `User`, de sorte qu'une fuite de la base de données n'expose jamais de jetons de rafraîchissement en clair [25].

La stratégie JWT est enregistrée comme guard global via `JwtGuard` (`guards/jwt.guard.ts`), qui étend `AuthGuard('jwt')` de Passport. Ce guard est appliqué sur tous les endpoints protégés du backend, garantissant que seules les requêtes portant un jeton d'accès valide et non expiré peuvent atteindre les handlers NestJS.

Rotation des jetons de rafraîchissement

La rotation des jetons de rafraîchissement (*refresh token rotation*) est un mécanisme de sécurité selon lequel chaque utilisation d'un jeton de rafraîchissement entraîne son invalidation immédiate et l'émission d'une nouvelle paire de jetons [26]. Ce mécanisme

est implémenté dans la méthode `refreshTokens()` de `auth.service.ts`, invoquée via l'endpoint `POST /auth/refresh`.

Lors d'un appel à cet endpoint, NestJS valide le jeton de rafraîchissement soumis en le comparant au hachage stocké en base de données. Si la vérification réussit, une nouvelle paire jeton d'accès / jeton de rafraîchissement est générée : le nouveau jeton de rafraîchissement est haché et remplace l'ancien en base, tandis que l'ancien est définitivement invalidé. Ce mécanisme garantit que la réutilisation d'un jeton de rafraîchissement volé soit détectable : si un attaquant tente d'utiliser un jeton déjà consommé, la vérification du hachage échoue et la session est révoquée [27].

Délégation OAuth GitHub au backend

Le flux OAuth 2.0 GitHub est entièrement délégué au backend NestJS plutôt que géré directement par NextAuth.js côté frontend. Ce choix architectural garantit que les tokens GitHub ne transitent jamais en clair dans le navigateur et que la logique de création ou de mise à jour du compte utilisateur reste dans la couche backend, où elle peut être auditée et testée indépendamment. La stratégie OAuth GitHub est implémentée dans `github-oauth.strategy.ts` via `PassportStrategy` et le package `passport-github2`.

Le flux se déroule en deux étapes. Lors de l'initiation, le frontend redirige l'utilisateur vers `GET /auth/github`, qui déclenche le guard `GithubOAuthGuard` (`guards/github-oauth.guard.ts`) et redirige vers le serveur d'autorisation GitHub avec les scopes requis. Après consentement de l'utilisateur, GitHub retourne un code d'autorisation vers l'URL de callback `GET /auth/github/callback`. NestJS échange ce code contre un token d'accès GitHub et invoque la méthode `validateOrCreateOAuthUser()` du service d'authentification, qui crée ou met à jour l'enregistrement `User` en base de données en associant le `githubId` retourné par GitHub au profil. NestJS émet ensuite une paire de jetons JWT propres à la plateforme et les retourne au frontend, de sorte que les appels API ultérieurs du client utilisent exclusivement les JWT Hostifer et non les tokens GitHub.

Rotation des jetons dans le callback NextAuth.js

Côté frontend Next.js, la gestion des sessions est assurée par NextAuth.js, configuré avec un `CredentialsProvider` personnalisé qui délègue la validation des identifiants au backend NestJS. NextAuth.js persiste les jetons JWT émis par le backend dans sa session chiffrée côté serveur, évitant toute exposition dans le `localStorage` du navigateur, vulnérable aux attaques XSS [28].

La rotation des jetons est orchestrée dans le `callback jwt` de NextAuth.js, qui est invoqué à chaque validation de session [?]. Ce callback implémente la logique suivante : si

la date d'expiration du jeton d'accès courant est dépassée, NextAuth.js appelle automatiquement l'endpoint `POST /auth/refresh` du backend NestJS en transmettant le jeton de rafraîchissement stocké en session. Si NestJS retourne une nouvelle paire de jetons, le callback met à jour la session avec les nouveaux jetons et la nouvelle date d'expiration, de manière totalement transparente pour l'utilisateur. En cas d'échec du rafraîchissement (jeton révoqué ou expiré), le callback positionne un champ `error: "RefreshAccessTokenError"` dans le token NextAuth.js, que le callback `session` propage au client, déclenchant une déconnexion forcée et une redirection vers la page de connexion [29].

Cette architecture à deux couches — rotation au niveau NestJS pour la sécurité de l'infrastructure, et rafraîchissement automatique au niveau NextAuth.js pour la transparence de l'expérience utilisateur — garantit des sessions longues sans compromettre la sécurité des jetons.

3.2.3 Module de déploiement et orchestration Argo Workflows

Le module `deployments/` constitue le cœur fonctionnel du backend Hostifer. Il est responsable de l'orchestration complète du cycle de vie des déploiements : soumission des workflows à Argo Workflows, synchronisation périodique de leur statut, et mise à jour transactionnelle des enregistrements `Deployment` et `DeploymentStep` en base de données PostgreSQL. Il expose également l'endpoint SSE de streaming des journaux de build, décrit dans la section suivante.

Soumission des workflows via l'API REST Argo

Argo Workflows expose un serveur API REST dédié (`argo-server`), distinct de l'API Kubernetes, qui offre des endpoints pour la soumission, l'interrogation et l'annulation des workflows [30]. La soumission d'un workflow depuis NestJS n'est donc pas effectuée via le client Kubernetes, mais via une requête HTTP `POST` vers l'endpoint `/api/v1/workflows/{namespace}/` de l'API Argo, en transmettant un corps JSON indiquant le *WorkflowTemplate* à instancier ainsi que les paramètres dynamiques du déploiement [31].

La méthode `submitDeployWorkflow(payload)` du service `argo.service.ts` construit ce corps de requête en injectant les paramètres propres à chaque déploiement : identifiant du projet, tag de l'image à construire, namespace Kubernetes du tenant, référence au dépôt Git et branche cible. L'authentification auprès de l'API Argo est assurée par un jeton de service (*ServiceAccount token*) transmis dans l'en-tête `Authorization: Bearer`. En retour, l'API Argo retourne le manifeste du *Workflow* instancié, dont le champ `metadata.name` est extrait et stocké dans le champ `argoWorkflowName` de l'enregistrement `Deployment` en base de données. Ce nom est utilisé comme clé d'identification pour

toutes les interrogations de statut ultérieures.

De manière symétrique, la méthode `submitDeleteWorkflow(payload)` soumet un *WorkflowTemplate* de suppression lors de la demande de déprovisionnement d'un projet (CU08), orchestrant la suppression de la release Helm, du namespace Kubernetes et des enregistrements DNS via les jobs Go dédiés.

Synchronisation du statut par polling

Argo Workflows ne propose pas de mécanisme de callback ou de webhook natif permettant à un système externe d'être notifié de l'évolution du statut d'un workflow. Le statut est exposé via l'endpoint `GET /api/v1/workflows/{namespace}/{name}` qui retourne l'objet *Workflow* complet, dont le champ `status.phase` reflète l'état courant : `Pending`, `Running`, `Succeeded`, `Failed` ou `Error` [30]. Le champ `status.nodes` contient quant à lui l'état détaillé de chaque nœud du workflow, correspondant à une étape individuelle du pipeline.

La synchronisation du statut est implémentée dans la méthode `syncDeploymentStatus(deploymentName)` de `deployments.service.ts`, selon un mécanisme de *polling* périodique. Cette méthode est invoquée régulièrement par le service NestJS après la soumission d'un workflow. À chaque appel, elle interroge l'API Argo via la méthode `getWorkflowStatus(workflowName)` du service `argo.service.ts`, parse la réponse JSON retournée, et en extrait les informations de statut nécessaires à la mise à jour de la base de données.

La correspondance entre les phases Argo et les statuts internes de Hostifer est définie comme suit. Tant que le workflow est en phase `Running`, le service examine l'état des nœuds individuels (`status.nodes`) pour déterminer quelle étape du pipeline est en cours d'exécution (*Build*, *Provision* ou *DNS*) et met à jour le statut du `Deployment` en conséquence (`BUILDING`, `PROVISIONING` ou `CONFIGURING_DNS`). Lorsque le workflow atteint la phase `Succeeded`, le statut est mis à jour à `LIVE` et le timestamp `finishedAt` est enregistré. En cas de phase `Failed` ou `Error`, le statut passe à `FAILED` et le message d'erreur est extrait via la méthode privée `extractWorkflowFailureMessage()`, qui parcourt les nœuds du workflow à la recherche du premier nœud en état d'échec et récupère son champ `message`. En cas d'annulation explicite, le statut passe à `CANCELLED`.

Mise à jour des enregistrements `Deployment` et `DeploymentStep`

Chaque appel à `syncDeploymentStatus()` se conclut par une mise à jour transactionnelle des enregistrements Prisma correspondant au déploiement. Deux entités sont mises à jour simultanément lors de chaque cycle de synchronisation.

L'entité `Deployment` est mise à jour avec le nouveau statut, le timestamp `updatedAt`,

et — lorsque le déploiement atteint un état terminal — les champs `finishedAt` et `totalDuration` calculés comme la différence en secondes entre `finishedAt` et `createdAt`.

L'entité `DeploymentStep` représente l'état individuel de chacune des trois étapes du pipeline (*Build*, *Provision*, *DNS*). Pour chaque étape, un enregistrement `DeploymentStep` est créé ou mis à jour avec le statut du nœud Argo correspondant, son heure de démarrage (`startedAt`), son heure de fin (`finishedAt`), et un message d'erreur optionnel. Cette granularité permet à l'interface utilisateur d'afficher une progression détaillée par étape, et non uniquement le statut global du déploiement.

Le flux complet de déclenchement d'un déploiement, depuis la réception de la requête `POST /deployments` jusqu'au passage en état `LIVE`, est illustré dans la Figure 2.5 (diagramme de séquence du déploiement).

TABEAU 3.4 – Correspondance entre les phases Argo Workflows et les statuts internes Hostifer

Phase Argo	Nœud actif	Statut (DeploymentStatus) Hostifer
Pending	—	QUEUED
Running	build	BUILDING
Running	provision	PROVISIONING
Running	dns	CONFIGURING_DNS
Succeeded	—	LIVE
Failed	—	FAILED
Error	—	FAILED
(annulé)	—	CANCELLED

3.2.4 Module de streaming des logs

Le streaming des journaux de build en temps réel constitue une fonctionnalité centrale de l'expérience utilisateur de Hostifer. Elle repose sur une architecture à deux composants distincts : l'endpoint SSE (*Server-Sent Events*) exposé par le contrôleur NestJS, et le générateur asynchrone de journaux Loki implémenté dans le service `loki.service.ts`. La séparation entre ces deux couches respecte le principe de responsabilité unique : le contrôleur gère le protocole de transport SSE, tandis que le service Loki encapsule toute la logique d'interrogation et de pagination des journaux.

Endpoint SSE avec le décorateur `@Sse()`

NestJS fournit un support natif du protocole SSE via le décorateur `@Sse()` du package `@nestjs/common` [32]. Ce protocole, standardisé par le W3C et nativement supporté par les navigateurs via l'API `EventSource`, permet au serveur de pousser des messages vers le client sur une connexion HTTP persistante unidirectionnelle, sans les surcoûts protocolaires des WebSockets [33]. Il est particulièrement adapté au cas d'usage du streaming de journaux, qui est par nature unidirectionnel (serveur $\rightarrow$ client) et ne nécessite pas de communication bidirectionnelle.

L'endpoint de streaming est déclaré dans `deployments.controller.ts` à la route `GET /deployments/:id/logs`, protégé par le `JwtGuard`. Le handler retourne un `Observable<MessageEvent>` de RxJS, que NestJS sérialise automatiquement en événements SSE conformes à la spécification `text/event-stream` [32]. Chaque événement transmis au client contient un champ `data` encapsulant un objet `LogLine` sérialisé en JSON, comprenant le contenu de la ligne de journal, son timestamp nanoseconde, et les labels Loki associés.

La conversion du générateur asynchrone retourné par `loki.service.ts` en `Observable` RxJS est réalisée via l'opérateur `from()` de RxJS, qui accepte nativement les itérables asynchrones. Le flux SSE est clos automatiquement lorsque le générateur asynchrone se termine, c'est-à-dire lorsque le déploiement atteint un état terminal ou lorsque Loki ne retourne plus de nouvelles entrées pour l'identifiant de build concerné.

Connexion WebSocket et endpoint Loki `/tail`

La collecte des journaux depuis Loki est implémentée dans la méthode `streamDeploymentLogs(deploymentId, options)` du service `loki.service.ts`, sous la forme d'un générateur asynchrone (`async function*`) qui produit des objets `LogLine` au fur et à mesure de leur disponibilité.

Loki expose un endpoint WebSocket `GET /loki/api/v1/tail` conçu pour le streaming de logs en temps réel, en contraste avec l'endpoint HTTP `GET /loki/api/v1/query_range` utilisé pour les requêtes temporelles historiques. L'endpoint `/tail` accepte une requête `LogQL` et maintient une connexion WebSocket persistante, diffusant chaque nouvelle ligne de journal au fur et à mesure qu'elle est ingérée par Loki. Cette approche élimine les délais de polling et réduit significativement la surcharge réseau par rapport à des requêtes HTTP répétées.

NestJS établit et gère une connexion WebSocket unique par flux de journaux (identifié par un `streamKey` dérivé de l'identifiant du déploiement). Cette gestion centralisée est assurée par le service `LokiTailPool`, qui implémente un pattern de pooling et de fan-out : plusieurs clients (*subscribers*) abonnés au même flux partagent une unique connexion WebSocket vers Loki, réduisant ainsi la consommation de ressources. Chaque *subscriber*

maintient sa propre file d'attente d'événements (*queue*) de sorte qu'un client lent ne bloque pas les autres.

La connexion WebSocket est établie paresseusement, c'est-à-dire uniquement lors du premier abonnement à un flux donné. Les frames de données reçus depuis Loki sont parsés et normalisés en objets `LogLine`, puis distribués de manière asynchrone à chaque subscriber via sa file de réception. En cas de déconnexion réseau, le service implémente une stratégie de reconnexion avec backoff exponentiel, garantissant la continuité du flux en cas de défaillance transitoire.

Requête LogQL et stratégie de filtrage

LogQL, le langage de requête de Loki, structure toute requête autour d'un sélecteur de flux de journaux (*log stream selector*) obligatoire, suivi d'un pipeline de traitement optionnel. Le sélecteur de flux est une chaîne de paires clé-valeur délimitée par des accolades, dont la combinaison unique identifie un flux de journaux.

La requête LogQL construite par `loki.service.ts` pour chaque déploiement est de la forme :

```
{build_id="<deployment_id>", container="main"}
```

Le label `build_id` est un label personnalisé attaché par Promtail lors de la collecte des journaux du pod Builder Job, via une règle de *relabeling* dans la configuration du DaemonSet Promtail. Il permet d'isoler précisément les journaux d'un déploiement donné parmi l'ensemble des logs du cluster [34]. Le label `container="main"` restreint la sélection au conteneur principal du pod, excluant les conteneurs système injectés par Argo Workflows (conteneur `wait`, conteneur `init`) dont les journaux sont de nature infrastructurelle et sans intérêt pour l'utilisateur final.

Cette stratégie de filtrage par labels indexés est préférable à un filtrage par pipeline (`|=`) appliqué après sélection d'un flux large, car l'application des filtres sur les labels indexés en amont du pipeline de traitement réduit significativement le volume de données à parcourir par Loki. En ciblant directement le flux `{build_id, container}`, la requête évite que Loki ne charge et ne filtre l'intégralité des journaux du namespace, ce qui serait coûteux dans un environnement multi-tenant à charge élevée.

Les objets `LogLine` normalisés produits par le service Loki contiennent le contenu de la ligne de journal, son timestamp en nanosecondes, et les labels Loki associés. Le générateur `yield` ces objets un par un vers le contrôleur, lequel les diffuse au navigateur via la connexion SSE.

Gestion du cycle de vie de la session WebSocket

La session WebSocket gérée par `LokiTailPool` termine son cycle de vie selon deux conditions distinctes. Premièrement, lorsque le statut du déploiement en base de données atteint un état terminal (`LIVE`, `FAILED` ou `CANCELLED`), indiquant que le pipeline Argo Workflows a conclu son exécution et qu’aucune nouvelle ligne de journal ne sera produite ; dans ce cas, le service interroge périodiquement la base de données et détecte cette transition pour fermer proprement la connexion. Deuxièmement, après une période d’inactivité détectée via le service `IntervalTicker`, qui contrôle les pauses prolongées entre les émissions de logs et émet un signal de heartbeat pour confirmer que le flux est toujours actif ; lors d’un silence prolongé sans détection de changement de statut, le générateur interprète cela comme la fin effective de la construction et termine.

Dans les deux cas, la terminaison du générateur asynchrone provoque la complétion de l’`Observable RxJS` côté contrôleur, ce qui entraîne automatiquement la fermeture propre de la connexion SSE. Côté backend, lorsque le dernier subscriber se désabonne (ou que tous les clients fermés leur connexion SSE), la session WebSocket `LokiTailSession` est supprimée du pool et la connexion WebSocket vers Loki est fermée, libérant les ressources associées.

3.2.5 Module de gestion des variables d’environnement avec Vault

3.2.6 Versioning et rollback des déploiements

3.3 Implémentation du builder Go

3.3.1 Structure du binaire

Le binaire `hostifer-builder` est organisé en cinq fichiers Go à responsabilités strictement séparées, complétés par les manifests de configuration et d’exécution Kubernetes. Cette organisation minimale reflète la philosophie du projet : un seul chemin d’exécution, sans framework, sans injection de dépendances, et sans état partagé entre les composants.

`main.go` constitue le point d’entrée du programme et l’unique orchestrateur du pipeline de build. Il commence par valider l’ensemble des variables d’environnement requises via la fonction `requireEnv()`, qui retourne une erreur immédiate si l’une d’elles est absente, évitant tout démarrage dans un état indéfini. Il instancie ensuite le logger structuré, applique un contexte Go avec un *timeout* global de vingt minutes sur l’ensemble du pipeline — au-delà duquel le Job est annulé et retourne un code de sortie non nul — puis enchaîne séquentiellement les trois étapes : clonage, préparation Railpack, et build

Buildkit. Le programme attend six variables d'environnement à l'exécution : `REPO_URL` (URL du dépôt à cloner), `IMAGE_NAME` (référence complète de l'image OCI à produire), `BUILDKIT_HOST` (adresse gRPC du daemon Buildkitd), `BUILD_ID` et `TENANT_ID` (identifiants utilisés pour la corrélation des journaux dans Loki), et optionnellement `WORK_DIR` (répertoire de travail, `/tmp/build` par défaut).

`git.go` encapsule la logique de clonage du dépôt source. La fonction `CloneRepo(ctx, repoURL, destDir, log)` supprime d'abord tout répertoire de travail existant, garantissant que les tentatives de relance du job démarrent depuis un état propre, puis exécute un clonage superficiel (`git clone -depth 1`) pour ne récupérer que le dernier commit de la branche cible. Cette approche réduit significativement le volume de données transférées pour les dépôts à historique long [35].

`railpack.go` est le fichier le plus volumineux du projet. Il prend en charge l'intégralité de l'interaction avec Railpack CLI : exécution de `railpack prepare`, gestion des tentatives en cas d'échec transitoire via `GenerateBuildWithRetry()`, et post-traitement du plan généré via `patchRailpackPlanRunAsUser()`. Ce fichier expose également les fonctions utilitaires `injectNodeMemoryFlag()` et `nodeMemoryLimitMb()`, décrites en détail dans la section suivante.

`buildkit.go` gère la connexion au daemon Buildkitd et l'exécution du build OCI. La fonction principale `BuildAndPush(ctx, buildkitHost, contextDir, imageName, log)` établit une connexion gRPC vers le daemon Buildkitd à l'adresse fournie par `BUILDKIT_HOST`, charge les credentials Docker depuis la configuration runtime du conteneur, soumet le contexte de build au frontend Railpack via l'API LLB de Buildkit, et pousse l'image résultante vers le registre interne de la plateforme [36]. La progression de la construction est capturée ligne par ligne par le type `buildkitProgressLogWriter` et émise vers le logger Zap, ce qui permet à Promtail de la collecter et de la transmettre à Loki pour le streaming SSE.

`logger.go` instancie un logger Zap en mode production via la fonction `NewLogger(buildID, tenantID string)`. Ce logger attache systématiquement les champs `build_id` et `tenant_id` à chaque entrée de journal émise, sous forme de JSON structuré sur `stdout`. Cette structuration est indispensable pour que les règles de *relabeling* Promtail puissent extraire ces valeurs comme labels Loki indexables, permettant le filtrage précis par déploiement décrit dans la section 3.2.4 [37].

L'image Docker du builder est construite en deux étapes (*multi-stage build*) : une première étape compile le binaire Go dans une image builder Go officielle, et une seconde étape produit l'image runtime minimaliste contenant uniquement le binaire compilé, `git`, `curl`, les certificats CA, et le binaire `railpack` installé dans `/usr/local/bin`. Une étape de préchauffage (*warmup*) Railpack est exécutée à la fin de la construction de l'image

pour s’assurer que le binaire est opérationnel dès le premier démarrage du Job, sans délai d’initialisation.

Le tableau 3.5 récapitule la structure du projet builder et le rôle de chaque fichier.

TABLEAU 3.5 – Structure du projet builder Go

Fichier	Type	Rôle principal
<code>main.go</code>	Point d’entrée	Validation des variables d’environnement, timeout global 20 min, orchestration du pipeline
<code>git.go</code>	Module	Clonage superficiel du dépôt Git (<code>-depth 1</code>), nettoyage du répertoire de travail
<code>railpack.go</code>	Module	Exécution de <code>railpack prepare</code> , gestion des tentatives (max. 3), patch du plan généré
<code>buildkit.go</code>	Module	Connexion gRPC à Buildkitd, build LLB via frontend Railpack, push vers le registre OCI
<code>logger.go</code>	Utilitaire	Initialisation du logger Zap avec champs <code>build_id</code> et <code>tenant_id</code>
Dockerfile	Image runtime	Build multi-stage : compilation Go + image runtime avec <code>git</code> , <code>railpack</code> , <code>curl</code>
<code>go.mod</code>	Dépendances	Dépendances : <code>moby/buildkit</code> , <code>docker/cli</code> , <code>uber/zap</code> , <code>grpc</code>

3.3.2 Détection de framework

La détection automatique du framework applicatif est entièrement déléguée à Railpack, un outil open source développé par Railway et successeur de Nixpacks [38]. La prise en charge des langages et frameworks dans Railpack est assurée par un système de *providers*, chacun associé à un langage spécifique (Node.js, Python, PHP, Go, Ruby, Java, etc.). Chaque provider analyse le dépôt source et détermine s’il correspond à son domaine en inspectant les fichiers présents à la racine du projet.

Le mécanisme de détection de Railpack repose sur l'analyse statique du système de fichiers du dépôt cloné, sans exécution de code applicatif. Railpack détecte automatiquement le langage (Node.js, Python, Go, PHP, Java, Ruby, etc.), le framework (Next.js, Laravel, Django, Spring Boot, etc.) et les dépendances via les fichiers de manifeste propres à chaque écosystème (`package.json`, `pyproject.toml`, `go.mod`, etc.).

Dans le builder Hostifer, la détection est déclenchée par l'invocation de la commande `railpack prepare <srcDir>` depuis la fonction `GenerateBuild()` de `railpack.go`. Cette commande produit deux fichiers dans le répertoire source : `railpack-plan.json`, qui contient le plan de build complet sérialisé en JSON, et `railpack-info.json`, qui contient les métadonnées de détection incluant le langage et le framework identifiés. Le plan de build est un objet JSON contenant l'ensemble des informations nécessaires à la génération de l'image, notamment les étapes de build (*steps*), les commandes à exécuter, les dépendances entre étapes et les instructions de mise en cache.

La commande `railpack prepare` est exécutée dans la fonction `runRailpack()`, qui lance le binaire Railpack comme sous-processus Go via `exec.CommandContext`, capture sa sortie standard et son flux d'erreur, et normalise le résultat en lignes de journal émises vers le logger Zap. Si Railpack retourne un code de sortie non nul, la fonction retourne une erreur Go qui déclenche le mécanisme de tentatives implémenté dans `GenerateBuildWithRetry()`. Ce mécanisme relance l'intégralité de la phase de préparation jusqu'à trois fois (`maxAttempts = 3`), en nettoyant les fichiers de sortie partiels entre chaque tentative, pour absorber les défaillances transitoires liées à la résolution de dépendances réseau ou à des conditions de concurrence dans l'environnement Kubernetes [38].

Un cas particulier est géré pour la compatibilité avec les versions antérieures de Railpack : si la première invocation échoue parce que le flag `-hide-pretty-plan` n'est pas supporté par la version installée dans l'image, `runRailpack()` relance la commande sans ce flag. Cette logique de *fallback* garantit la compatibilité du builder avec différentes versions du binaire Railpack sans nécessiter de mise à jour de l'image.

Le résultat de la détection — le framework identifié — est extrait du fichier `railpack-info.json` par le backend NestJS après la fin du workflow Argo, et persisté dans le champ `framework` de l'entité `Project` en base de données PostgreSQL. Cette information est ensuite affichée dans l'interface utilisateur sur la page de détail du projet, offrant à l'utilisateur une confirmation visuelle que son framework a bien été reconnu automatiquement.

3.3.3 Génération du plan Railpack

La génération du plan de build constitue la phase la plus déterminante du pipeline de construction, car elle produit la spécification complète de l'image OCI à construire. Elle se

déroule en deux temps : l’invocation de Railpack CLI pour la détection et la génération du plan brut, suivie d’un post-traitement du plan JSON par le builder Go pour y appliquer des contraintes de sécurité et d’optimisation mémoire.

Invocation de railpack prepare

La génération du plan est déclenchée par la méthode `GenerateBuild(ctx, srcDir, log)` de `railpack.go`, qui invoque la commande `railpack prepare <srcDir>` via `exec.CommandContext`. Cette commande produit deux fichiers en sortie : `railpack-plan.json`, qui contient le plan de build sérialisé, et un fichier d’information `railpack-info.json`. Le plan de build est un objet JSON contenant l’ensemble des informations nécessaires à la génération de l’image, notamment les étapes (*steps*), les commandes à exécuter séquentiellement au sein de chaque étape, les dépendances entre étapes, et les directives de mise en cache permettant à Buildkit d’exécuter les étapes indépendantes en parallèle.

La version du binaire Railpack à utiliser est contrôlable via la variable d’environnement `RAILPACK_VERSION`. La référence de l’image frontend Railpack à transmettre à Buildkit est construite par la fonction `railpackFrontendImage()` à partir de cette version, garantissant la cohérence entre la version CLI ayant généré le plan et la version de l’image frontend interprétant ce même plan lors de la construction.

L’ensemble de la phase de préparation est encapsulée dans `GenerateBuildWithRetry(ctx, srcDir, log, maxAttempts)`, qui relance l’intégralité du processus en cas d’échec, jusqu’à un maximum de trois tentatives. Entre chaque tentative, les fichiers de sortie partiels (`railpack-plan.json`, `railpack-info.json`) sont supprimés afin d’éviter que Buildkit consomme un plan corrompu ou incomplet lors d’une tentative ultérieure.

Patch du plan JSON pour UID 1000

À l’issue de la génération réussie du plan, la fonction `patchRailpackPlanRunAsUser(srcDir, log)` lit, modifie et réécrit le fichier `railpack-plan.json`. Ce patch poursuit trois objectifs de sécurité et de compatibilité.

Premièrement, le plan est modifié pour forcer l’exécution du processus applicatif sous l’UID 1000 et le GID 1000. Par défaut, Railpack peut générer des images dont le processus s’exécute en tant que `root` (UID 0), ce qui constitue une mauvaise pratique de sécurité dans un environnement multi-tenant et est incompatible avec les *PodSecurityPolicies* restrictives définies par Kubernetes sur certains clusters [39]. L’exécution sous un UID non privilégié réduit la surface d’attaque en cas de faille applicative permettant une évvasion de conteneur.

Deuxièmement, une étape `chown -R 1000:1000 /app` est injectée dans le plan lorsque

la structure de celui-ci le permet, afin de transférer la propriété du répertoire applicatif à l'UID cible avant le démarrage du processus. Cette étape est nécessaire car les outils de build (gestionnaires de paquets, compilateurs) opèrent généralement en tant que `root` pendant la phase de construction ; sans ce transfert de propriété, le processus en UID 1000 ne pourrait pas écrire dans son propre répertoire de travail à l'exécution.

Troisièmement, les commandes de démarrage (*start commands*) du plan sont inspectées pour détecter les invocations de `node`, `npm` ou `yarn`. Si une telle invocation est détectée, le flag `NODE_OPTIONS` est injecté via la fonction `injectNodeMemoryFlag()`, décrite ci-après.

Injection de `NODE_OPTIONS` pour la gestion mémoire

L'injection de la limite de tas mémoire Node.js répond à un problème spécifique aux environnements conteneurisés sous Kubernetes. Sans configuration explicite, le moteur JavaScript V8 de Node.js ne connaît pas les limites mémoire du conteneur et ne déclenche pas le garbage collector de manière agressive lorsque la consommation approche la limite du conteneur, ce qui provoque un crash OOM (*Out-of-Memory Kill*) du conteneur par le noyau Linux avant que V8 n'ait pu libérer de la mémoire.

La solution implémentée dans `railpack.go` consiste à injecter la variable d'environnement `NODE_OPTIONS` avec la valeur `-max-old-space-size=<limitMb>` dans la commande de démarrage du plan Railpack. L'utilisation de `NODE_OPTIONS` est préférée à l'injection directe du flag dans la commande `node`, car elle est prise en compte par tous les processus Node.js enfants créés dans l'environnement, y compris ceux démarrés par `npm` ou `yarn`.

La valeur de la limite en mégaoctets est déterminée par la fonction `nodeMemoryLimitMb()`, qui lit la variable d'environnement `MEMORY_LIMIT_MB` exposée au conteneur par le manifest Kubernetes du Job via le mécanisme `resourceFieldRef` de Kubernetes. Cette approche rend la limite dynamique et proportionnelle aux ressources allouées au Job par le plan tarifaire du tenant, sans hard-coding d'une valeur fixe dans l'image. Node.js depuis la version 12 est dit *container-aware* et peut utiliser les limites cgroups du conteneur, mais l'injection explicite de `-max-old-space-size` reste recommandée pour découpler précisément la limite du tas V8 de la limite mémoire totale du conteneur. En l'absence de la variable `MEMORY_LIMIT_MB`, la fonction retourne une valeur par défaut conservative afin de garantir un comportement prévisible même sans configuration explicite.

FIGURE 3.3 – Extrait du plan `railpack-plan.json` généré pour une application Node.js avec les modifications appliquées

3.3.4 Construction et push de l'image via Buildkit

La construction de l'image OCI et son push vers le registre interne constituent la dernière étape du pipeline de build. Elle est entièrement implémentée dans `buildkit.go` via le client Go officiel de Buildkit, exposé par le package `github.com/moby/buildkit/client` [36].

Connexion au daemon Buildkitd

Buildkit est déployé sur le cluster Kubernetes en tant que *Deployment* avec un pod Buildkitd persistant, accessible depuis les autres pods du cluster via une adresse de service Kubernetes. Le daemon Buildkitd expose son API gRPC sur un socket TCP, permettant aux clients distants de lui soumettre des requêtes de build via une connexion réseau standard.

La fonction `BuildAndPush(ctx, buildkitHost, contextDir, imageName, log)` établit la connexion au daemon via `client.New(ctx, buildkitHost)`, où `buildkitHost` est la valeur de la variable d'environnement `BUILDKIT_HOST`. Cette variable est injectée dans le Job par le ConfigMap Kubernetes `build-configmap`, qui contient l'adresse du service Buildkitd au format `tcp://<service>:<port>`. La connexion est établie de manière asynchrone avec gestion des tentatives de reconnexion intégrée au client Buildkit.

Les credentials Docker nécessaires pour pousser l'image vers le registre interne sont chargés depuis la configuration Docker runtime du conteneur via le package `github.com/docker/cli/com` puis transmis à Buildkit sous forme de `session.AuthProvider` attaché à la requête de build. Ce mécanisme de délégation d'authentification évite que les credentials ne soient exposés dans les logs ou dans les variables d'environnement du Job.

Résolution du frontend Railpack via gateway.v0

Buildkit distingue deux types de frontends pour l'interprétation des définitions de build [40]. Le frontend `dockerfile.v0` est le frontend officiel pour les Dockerfiles, tandis que le frontend `gateway.v0` est un frontend générique permettant d'utiliser n'importe quelle image OCI comme interpréteur de build, en lui déléguant la conversion de la définition de build en LLB (*Low-Level Build*).

Le builder Hostifer utilise le frontend `gateway.v0` en lui fournissant l'image frontend Railpack référencée par `railpackFrontendImage()` comme source d'interprétation. Buildkit télécharge cette image depuis GHCR si elle n'est pas déjà présente dans son cache local, l'instancie, et lui transmet le répertoire source du dépôt cloné ainsi que le fichier `railpack-plan.json` patché. Le frontend Railpack interprète ce plan et génère le

graphe de dépendances LLB correspondant, que Buildkit exécute pour produire l'image OCI finale. LLB (*Low-Level Build*) est le format intermédiaire binaire de Buildkit, défini comme un protocol buffer, qui représente le graphe de dépendances des opérations de build sous forme d'un DAG (*Directed Acyclic Graph*) permettant l'exécution parallèle des étapes indépendantes et la réutilisation du cache de couches.

L'image résultante est poussée directement vers le registre interne OCI de la plateforme via l'option d'export `type=image,name=<imageName>,push=true,registry.insecure=true`. Le flag `registry.insecure=true` est nécessaire car le registre interne de la plateforme est exposé en HTTP sans TLS au sein du cluster, selon le modèle de registre interne non authentifié courant dans les environnements Kubernetes isolés [36].

Streaming des journaux vertex par vertex

Le suivi de la progression de la construction est assuré par le type `buildkitProgressLogWriter`, implémenté dans `buildkit.go`. Ce type implémente l'interface `io.Writer` et est passé à la fonction de solve Buildkit comme récepteur de la progression. Buildkit émet sa progression sous la forme d'une séquence de *vertex* — chaque vertex correspondant à une étape atomique du graphe LLB (installation d'un package, exécution d'une commande, copie de fichiers, etc.) — accompagnés de leurs logs de sortie standard.

La méthode `Write(p []byte)` reçoit ces données de progression brutes, les accumule dans un buffer interne, et émet chaque ligne complète (délimitée par `\n`) vers le logger Zap via `log.Info()`. La méthode `Flush()` est appelée à la fin du build pour s'assurer qu'aucune ligne partielle ne reste dans le buffer. La fonction utilitaire `stripTrailingCR()` normalise les fins de ligne en supprimant les caractères `\r` résiduels produits par certains outils de build qui utilisent le retour chariot pour l'affichage de barres de progression en terminal.

Chaque ligne émise par le logger Zap est collectée par Promtail, qui l'indexe dans Loki avec les labels `build_id` et `tenant_id` attachés par le logger. Ces journaux deviennent ainsi immédiatement disponibles pour le streaming SSE décrit dans la section 3.2.4, offrant à l'utilisateur une visibilité en temps réel sur chaque étape de construction de son image OCI.

3.4 Implémentation de l'infrastructure Kubernetes

L'infrastructure Kubernetes de Hostifer est entièrement décrite sous forme déclarative et provisionnée de manière automatisée. Chaque composant — qu'il s'agisse de l'isolation d'un tenant, du pipeline d'orchestration Argo Workflows, de la collecte des journaux ou

de la gestion des certificats TLS — est défini par des manifests versionnés et déployés via GitOps. Cette section décrit l'implémentation concrète de chacun de ces composants.

3.4.1 Chart Helm multi-tenant

Le chart Helm `tenant-chart` est la pièce centrale du modèle de provisionnement de Hostifer. Chaque déploiement d'application utilisateur déclenche l'invocation de ce chart via l'étape *Provision* du pipeline Argo Workflows, qui crée ou met à jour une release Helm dédiée au tenant concerné. L'ensemble des ressources Kubernetes nécessaires à l'isolation, à l'exposition et à la supervision du workload applicatif sont instanciées par ce seul chart, garantissant la reproductibilité et l'atomicité du provisionnement [41].

Le modèle retenu pour Hostifer est la multi-location basée sur les namespaces (*namespace-based multi-tenancy*), combinée à des NetworkPolicies pour restreindre les communications inter-namespaces. Ce modèle offre un bon équilibre entre isolation et efficacité opérationnelle sur un cluster partagé.

Ressources provisionnées par le chart

Namespace. Une ressource `Namespace` est créée avec le nom exact de l'identifiant du tenant (`.Values.tenant.id`), labellisée avec `hostifer.io/tenant: <tenant-id>` et `hostifer.io/managed: "true"`. Ce namespace constitue la frontière d'isolation primaire du tenant et le périmètre d'application de toutes les politiques de sécurité et de ressources.

ServiceAccount. Un `ServiceAccount` nommé `tenant-sa` est créé avec `automountServiceAccountToken: false`. Ce réglage empêche le montage automatique du token d'API Kubernetes dans les pods du tenant, réduisant la surface d'attaque en cas de compromission applicative : un pod compromis ne peut pas, par défaut, interagir avec l'API Kubernetes du cluster [39].

Deployment. Un `Deployment` nommé `<tenant-id>-app` orchestre les pods applicatifs du tenant. Le pod template applique un contexte de sécurité strict : `runAsNonRoot: true`, `runAsUser: 1000`, `fsGroup: 1000`, et `automountServiceAccountToken: false`. Ces valeurs sont cohérentes avec le patch UID appliqué par le builder Go lors de la génération du plan Railpack (section 3.3.3). Les ressources CPU et mémoire du conteneur (`requests` et `limits`) sont directement mappées depuis les valeurs `.Values.plan.cpu` et `.Values.plan.memory`, garantissant que les ressources allouées correspondent au plan tarifaire souscrit par le tenant. Une contrainte de répartition topologique (`topologySpreadConstraints`) avec `maxSkew: 1` sur le label `kubernetes.io/hostname` assure la distribution équitable des réplicas sur les nœuds disponibles du cluster, améliorant la résilience aux défaillances de nœuds. Des sondes de *readiness* et de *liveness* HTTP `GET /` sont configurées avec des

délais initiaux respectifs de 5 et 15 secondes, permettant à Kubernetes de détecter et de redémarrer automatiquement les pods non fonctionnels.

Service. Un **Service** de type **ClusterIP** nommé `tenant-svc` expose le port 80 vers le port applicatif configurable (`.Values.app.port`, défaut 3000). Il sélectionne les pods via les labels `app: tenant-app` et `hostifer.io/tenant: <tenant-id>`, garantissant que chaque service ne route le trafic que vers les pods appartenant au même tenant.

IngressRoute (Traefik CRD). Une ressource **IngressRoute** de l'API `traefik.io/v1alpha1` nommée `tenant-ingress` expose l'application sur Internet. La règle de routage `Host(<tenant-subdomain>)` achemine le trafic entrant sur le port 80 vers le service `tenant-svc:80`. Cette ressource dépend de l'installation préalable des CRDs Traefik sur le cluster et de l'existence de l'entryPoint `web` dans la configuration Traefik.

LimitRange. Une ressource **LimitRange** nommée `tenant-limits` applique des enveloppes de ressources par conteneur dans le namespace. Elle définit des valeurs par défaut pour les `requests` et `limits` CPU et mémoire (tirées des valeurs du plan), un maximum égal aux limites du plan, et des minimums absolus de 10m CPU et 16Mi mémoire. Le **LimitRange** est un mécanisme complémentaire au **ResourceQuota** qui empêche la monopolisation des ressources par un objet individuel au sein d'un namespace, en s'assurant que chaque pod dispose d'une allocation minimale viable et ne dépasse pas le plafond défini.

ResourceQuota. Une ressource **ResourceQuota** nommée `tenant-quota` plafonne la consommation agrégée de ressources dans le namespace. Le **ResourceQuota** limite la consommation totale d'un namespace en définissant des plafonds durs sur les `requests.cpu`, `requests.memory`, `limits.cpu` et `limits.memory`, ainsi que sur le nombre total de pods. Le chart fixe également à zéro le quota de `persistentvolumeclaims`, interdisant explicitement la création de volumes persistants par les applications stateful — cohérent avec le positionnement de Hostifer sur les applications stateless HTTP. Les compteurs d'objets sont aussi bornés : 10 **Secrets** et 10 **ConfigMaps** maximum par namespace, prévenant les abus de l'API Kubernetes par un tenant malveillant.

NetworkPolicies (×4). Le chart crée quatre **NetworkPolicies** dont l'action combinée implémente un modèle *allowlist* strict [42]. La séquence correcte de durcissement d'un namespace multi-tenant commence par la création d'une **NetworkPolicy** *default-deny*, puis l'ajout de règles d'autorisation explicites pour chaque flux légitime. Les quatre politiques sont les suivantes :

1. `default-deny-all` — appliquée à tous les pods du namespace (`podSelector: {}`), elle bloque l'intégralité du trafic entrant et sortant par défaut, en spécifiant les types **Ingress** et **Egress** sans aucune règle d'autorisation. C'est le socle de la politique

d'isolement inter-tenant.

2. **allow-traefik-ingress** — autorise le trafic entrant uniquement depuis les pods situés dans le namespace Traefik, identifié par le label `kubernetes.io/metadata.name = .Values.traefik.namespace`. Cette règle garantit que les pods applicatifs ne sont joignables que via l'Ingress Controller et non directement depuis d'autres namespaces.
3. **allow-dns-egress** — autorise le trafic sortant vers les ports UDP/53 et TCP/53 pour permettre la résolution DNS depuis les pods applicatifs, indispensable à toute connexion réseau sortante.
4. **allow-internet-egress** — autorise le trafic sortant vers 0.0.0.0/0 en excluant les plages RFC1918 (10.0.0.0/8, 172.16.0.0/12, 192.168.0.0/16). Cette règle permet aux applications tenantes d'appeler des APIs externes sur Internet tout en bloquant les tentatives d'accès aux ressources internes du cluster ou du réseau privé de l'hébergeur, renforçant l'isolation réseau inter-tenant.

Schéma de valeurs et convention de nommage

Le chart est entièrement paramétrable via un fichier `values.yaml` structuré en quatre blocs : **tenant** (identifiant, sous-domaine, domaine), **app** (image OCI, port, réplicas, variables d'environnement), **plan** (ressources CPU/mémoire et quotas namespace), et **traefik** (namespace de l'Ingress Controller). Les ressources Kubernetes adoptent une convention de nommage explicite et déterministe : le Deployment est nommé `<tenant-id>-app`, le Service `tenant-svc`, le ServiceAccount `tenant-sa`, et le Namespace prend exactement la valeur de `tenant.id`. Cette convention permet à Argo Workflows et au backend NestJS de référencer les ressources par leur nom prévisible sans avoir à interroger l'API Kubernetes pour les découvrir.

Le tableau 3.6 récapitule l'ensemble des ressources créées par le chart et leurs paramètres clés.

TABLEAU 3.6 – Ressources Kubernetes provisionnées par le chart `tenant-chart`

Kind	Nom	Rôle et paramètres clés
Namespace	<code><tenant-id></code>	Frontière d'isolation primaire du tenant ; labels <code>hostifer.io/tenant</code> et <code>hostifer.io/managed</code>

Suite à la page suivante

Kind	Nom	Rôle et paramètres clés
ServiceAccount	tenant-sa	Compte de service sans montage automatique de token API (automountServiceAccountToken: false)
Deployment	<tenant-id>-app	Workload applicatif; UID 1000, runAsNonRoot, topologySpreadConstraints, sondes readiness/liveness
Service	tenant-svc	Endpoint ClusterIP stable; port 80 → app.port
IngressRoute	tenant-ingress	Route Traefik Host(<subdomain>.<domain>) → tenant-svc:80
LimitRange	tenant-limits	Enveloppe ressources par conteneur; min CPU 10m, min RAM 16Mi, max = limites du plan
ResourceQuota	tenant-quota	Plafonds agrégés namespace; PVC = 0, Secrets ≤ 10, ConfigMaps ≤ 10
NetworkPolicy	default-deny-all	Refus total du trafic entrant et sortant par défaut
NetworkPolicy	allow-traefik-ingress	Entrée autorisée depuis le namespace Traefik uniquement
NetworkPolicy	allow-dns-egress	Sortie autorisée vers DNS UDP/TCP 53
NetworkPolicy	allow-internet-egress	Sortie autorisée vers Internet public; RFC1918 bloquées
Pod (test hook)	<release>-test-validation	Validation post-install : wget HTTP vers tenant-svc depuis l'intérieur du namespace

FIGURE 3.4 – Extrait du template NetworkPolicy illustrant l’isolation inter-tenant et l’autorisation Traefik uniquement

3.4.2 Pipeline Argo Workflows

Argo Workflows est déployé sur le cluster ACK dans le namespace `hostifer-system` via son chart Helm officiel. Sa configuration est ajustée via un fichier `values.yaml` dédié qui restreint le périmètre d’exécution du contrôleur au seul namespace `hostifer-system` (`workflowNamespaces: [hostifer-system]`), active le mode d’authentification client (`-auth-mode=client`) pour l’accès au serveur API Argo, et expose le serveur Argo en mode `ClusterIP` sans exposition publique directe [43]. Deux `WorkflowTemplates` distincts définissent les pipelines de déploiement et de suppression de la plateforme.

WorkflowTemplate de déploiement : `hostifer-deploy`

Le `WorkflowTemplate` nommé `hostifer-deploy` orchestre le cycle de vie complet d’un déploiement applicatif en trois étapes séquentielles. Il est déclenché par le backend NestJS via une requête `POST` à l’API REST Argo, avec injection des paramètres dynamiques propres à chaque déploiement.

Paramètres dynamiques. Le template déclare cinq paramètres d’entrée injectés par le backend NestJS à chaque invocation : `repo_url` (URL du dépôt Git à construire), `subdomain` (sous-domaine public de l’application), `port` (port d’écoute de l’application), `tenant_id` (identifiant unique du tenant, utilisé comme nom de release Helm et de namespace), et `image_name` (référence complète de l’image OCI à construire et déployer). Ces paramètres sont référencés dans les templates de chaque étape via la syntaxe `{{workflow.parameters.<name>}}` garantissant l’isolation complète des données entre workflows concurrents.

Politiques de cycle de vie. Le `WorkflowTemplate` configure deux politiques de gestion du cycle de vie des pods et des workflows. La politique `podGC` avec la stratégie `OnPodCompletion` déclenche la suppression des pods immédiatement après leur complétion, libérant les ressources du cluster sans attendre [44]. La stratégie `tTlStrategy` définit des durées de rétention différenciées : 0 seconde après succès (suppression immédiate du workflow), 3600 secondes (1 heure) après échec pour permettre l’inspection des journaux, et 10 secondes pour toute autre complétion.

Étape 1 — Build. L’étape *Build* exécute le conteneur `ghcr.io/hostifer/builder-job:latest` décrit dans la section 3.3. Les variables d’environnement requises par le binaire Go (`REPO_URL`, `IMAGE_NAME`, `BUILDKIT_HOST`, `BUILD_ID`, `TENANT_ID`) sont injectées depuis les paramètres du workflow, à l’exception de `BUILDKIT_HOST` qui est codé en dur avec l’adresse du service Buildkitd interne au cluster (`tcp://buildkitd.hostifer-system.svc.cluster.local:1234`).

Les ressources allouées au pod de build sont dimensionnées pour absorber les workloads de compilation intensifs : 500m CPU en requête et 2 cœurs en limite, 512 Mi mémoire en requête et 2 Gi en limite.

Étape 2 — Provision. L'étape *Provision* utilise l'image officielle `alpine/helm:3.14.0` pour invoquer `helm install` avec le chart OCI `oci://ghcr.io/hostifer/tenant-chart`. Les paramètres Helm sont passés via des flags `-set` construits à partir des paramètres du workflow (`tenant.id`, `tenant.subdomain`, `app.image`, `app.port`). Le flag `-create-namespace` délègue la création du namespace à Helm, le flag `-wait` maintient le pod *Provision* en cours d'exécution jusqu'à ce que le `Deployment` du tenant soit en état `Ready`, et `-timeout 2m` impose une limite de temps pour éviter les blocages indéfinis. Les credentials GHCR pour le pull du chart OCI sont montés depuis le `Secret ghcr-login-secret` via un volume `docker-config`, copié dans le répertoire de configuration Helm (`/root/.config/helm/registry/config.json`) au démarrage du conteneur.

Étape 3 — DNS. L'étape *DNS* exécute le conteneur `ghcr.io/hostifer/create-dns-job:latest`, un binaire Go dédié à la création de l'enregistrement DNS du sous-domaine de l'application déployée. Le sous-domaine est passé comme argument positionnel au conteneur. Les credentials Cloudflare (token API, zone ID) sont injectés via `envFrom` depuis le `Secret cloudflare-secret`, évitant toute exposition de credentials dans les manifests versionnés.

WorkflowTemplate de suppression : `hostifer-delete`

Le `WorkflowTemplate hostifer-delete` orchestre la suppression complète des ressources d'un tenant. Contrairement au pipeline de déploiement dont les étapes sont séquentielles, les trois étapes de suppression sont exécutées **en parallèle** pour minimiser le temps total de déprovisionnement.

Étape 1 — Uninstall (parallèle). Invoque `helm uninstall <release_name>` avec `-wait` et `-timeout 2m`, supprimant l'ensemble des ressources Kubernetes créées par la release Helm du tenant (`Deployment`, `Service`, `IngressRoute`, `NetworkPolicies`, `ResourceQuota`, `LimitRange`, `ExternalSecret`, `ServiceAccount`).

Étape 2 — Delete DNS (parallèle). Exécute le conteneur `ghcr.io/hostifer/delete-dns-job:latest` pour supprimer l'enregistrement DNS Cloudflare associé au sous-domaine du tenant, avec les mêmes credentials injectés via `envFrom`.

Étape 3 — Delete Registry Image (parallèle). Exécute un conteneur `alpine:3.20` avec un script shell qui résout le digest SHA256 de l'image OCI du tenant dans le registre interne via l'API HTTP Registry v2, puis émet une requête `DELETE` pour supprimer le manifest correspondant. Cette étape libère l'espace disque du registre interne après suppression du tenant.

Handler onExit. Le template `nextjs-reconcile` est déclaré comme handler `onExit` du pipeline de suppression. Il est exécuté systématiquement à l'issue du workflow, qu'il soit en succès ou en échec, et déclenche une réconciliation de l'état affiché dans le frontend Next.js pour refléter la suppression effective du projet.

FIGURE 3.5 – Capture d'écran de l'interface Argo Workflows UI montrant un workflow de déploiement complet avec les trois étapes

3.4.3 Configuration de l'observabilité (Loki + Promtail)

3.4.4 Gestion des certificats TLS avec cert-manager

3.4.5 Collecte des métriques avec Prometheus

3.5 Présentation des interfaces utilisateur

3.5.1 Page d'accueil (Landing Page)

FIGURE 3.6 – Capture d'écran de la section Hero de la landing page

FIGURE 3.7 – Capture d'écran de la section Pricing

FIGURE 3.8 – Capture d’écran de la section comparaison concurrentielle

FIGURE 3.9 – Capture d’écran du dashboard avec liste des projets et statuts

3.5.2 Tableau de bord utilisateur

3.5.3 Assistant de déploiement (Deploy Wizard)

3.5.4 Gestion des variables d’environnement

3.5.5 Interface d’administration Argo Workflows

3.6 Tests et validation

3.6.1 Tests fonctionnels

3.6.2 Tests d’isolation multi-tenant

3.6.3 Tests de charge et performance

3.6.4 Tests de conformité réglementaire

3.7 Bilan et perspectives

3.7.1 Fonctionnalités réalisées

3.7.2 Difficultés rencontrées

3.7.3 Perspectives d’évolution

FIGURE 3.10 – Capture d’écran de la page de détail d’un projet

FIGURE 3.11 – Capture d’écran de l’étape 1 : validation de l’URL GitHub et détection du framework

FIGURE 3.12 – Capture d’écran de l’étape 2 : configuration (région, plan, variables d’environnement)

FIGURE 3.13 – Capture d’écran de l’étape 3 : streaming des logs de build en temps réel

FIGURE 3.14 – Capture d’écran de l’étape 4 : confirmation du déploiement réussi avec URL live

FIGURE 3.15 – Capture d’écran de l’interface de gestion des variables d’environnement avec masquage des secrets

FIGURE 3.16 – Capture d’écran de l’interface Argo Workflows montrant les étapes Build, Provision, DNS d’un déploiement réel

TABLEAU 3.7 – Résultats des tests de déploiement par framework (Express, NestJS, Next.js, Flask, Laravel) : statut, durée de build, mémoire consommée

TABLEAU 3.8 – Résultats des tests d’isolation réseau (tentative de connexion inter-tenant, résultat attendu vs observé)

FIGURE 3.17 – Graphique du temps de déploiement moyen par framework (de la soumission du workflow à l’état LIVE)

TABLEAU 3.9 – Tableau de performance : temps de build moyen, latence SSE, consommation mémoire buildkitd pour 1, 3 et 5 builds simultanés, plus métriques d’autoscaling (temps de montée HPA, recommandations VPA, latence d’ajout de nœuds)

TABLEAU 3.10 – Checklist de conformité (critère, mécanisme technique de conformité, statut vérifié)

TABLEAU 3.11 – Tableau de couverture fonctionnelle (besoin identifié au chapitre 2, statut d’implémentation : réalisé / partiel / reporté)

Conclusion Générale

Synthèse des contributions

Contribution technique

Contribution économique

Contribution sociétale

Perspectives d'évolution

Ouverture

Bibliographie

- [1] Object Management Group. Unified Modeling Language Specification, Version 2.5.1. OMG Document, 2017. [Online]. Available : <https://www.omg.org/spec/UML/2.5.1/PDF>. [Accessed : May 2025].
- [2] G. Booch, J. Rumbaugh, and I. Jacobson. *The Unified Modeling Language User Guide*. Addison-Wesley, 2 edition, 2005.
- [3] I. Sommerville. *Software Engineering*. Pearson, 10 edition, 2016.
- [4] The Kubernetes Authors. Kubernetes concepts. Kubernetes Documentation, 2024. [Online]. Available : <https://kubernetes.io/docs/concepts/>. [Accessed : May 2025].
- [5] K. S. Ahmad and A. Sana. Fuzzy_moscow : A fuzzy based moscow method for the prioritization of software requirements. In *2018 International Conference on Computing, Mathematics and Engineering Technologies (iCoMET)*, pages 1–5, 2018.
- [6] R. Yahyapour et al. Multi-tenancy : Deep dive on how cloud platforms serve many customers. *World Journal of Advanced Research and Reviews*, 26(1), 2025. [Online]. Available : https://journalwjarr.com/sites/default/files/fulltext_pdf/WJARR-2025-1608.pdf.
- [7] The Kubernetes Authors. Autoscaling workloads. Kubernetes Documentation, 2025. [Online]. Available : <https://kubernetes.io/docs/concepts/workloads/autoscaling/>. [Accessed : May 2025].
- [8] The Kubernetes Authors. Horizontal pod autoscaling. Kubernetes Documentation, 2025. [Online]. Available : <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>. [Accessed : May 2025].
- [9] Kubecost Team. The guide to kubernetes VPA by example. Kubecost Blog, 2025. [Online]. Available : <https://www.kubecost.com/kubernetes-autoscaling/kubernetes-vpa/>. [Accessed : May 2025].
- [10] Ksolves. Kubernetes autoscaling : HPA vs. VPA vs. cluster autoscaler. Ksolves Blog, 2025. [Online]. Available : <https://www.ksolves.com/blog/devops/>.

- [kubernetes-autoscaling-hpa-vs-vpa-vs-cluster-autoscaler](#). [Accessed : May 2025].
- [11] Microsoft. Tenancy models for a multitenant solution. Azure Architecture Center, Microsoft Learn, 2024. [Online]. Available : <https://learn.microsoft.com/en-us/azure/architecture/guide/multitenant/considerations/tenancy-models>. [Accessed : May 2025].
- [12] Alibaba Cloud. What is container service for kubernetes (ACK). Alibaba Cloud Documentation, 2024. [Online]. Available : <https://www.alibabacloud.com/help/en/ack/ack-managed-and-ack-dedicated/product-overview/what-is-ack>. [Accessed : May 2025].
- [13] Alibaba Cloud. Best practices for high availability stability of ACK. Alibaba Cloud Community Blog, 2024. [Online]. Available : <https://www.alibabacloud.com/blog/best-practices-for-high-availability-stability-of-alibaba-cloud-container-service-601779>. [Accessed : May 2025].
- [14] Alibaba Cloud. Internet access — virtual private cloud. Alibaba Cloud Documentation, 2024. [Online]. Available : <https://www.alibabacloud.com/help/en/vpc/use-cases/select-a-product-to-gain-access-to-the-internet>. [Accessed : May 2025].
- [15] Alibaba Cloud. Plan the network architecture for a managed cluster. Alibaba Cloud Documentation, 2025. [Online]. Available : <https://www.alibabacloud.com/help/en/ack/ack-managed-and-ack-dedicated/user-guide/kubernetes-cluster-network-planning>. [Accessed : May 2025].
- [16] Alibaba Cloud. What is application load balancer (ALB). Alibaba Cloud Documentation, 2025. [Online]. Available : <https://www.alibabacloud.com/help/en/slb/application-load-balancer/product-overview/what-is-alb>. [Accessed : May 2025].
- [17] Alibaba Cloud. Associate and disassociate an EIP with cloud resources. Alibaba Cloud Documentation, 2025. [Online]. Available : <https://www.alibabacloud.com/help/en/eip/bind-an-eip-to-a-cloud-resource>. [Accessed : May 2025].
- [18] Kubernetes SIGs. CSI plugin for kubernetes — alibaba cloud EBS/NAS/OSS. GitHub Repository, kubernetes-sigs/alibaba-cloud-csi-driver, 2024. [Online]. Available : <https://github.com/kubernetes-sigs/alibaba-cloud-csi-driver>. [Accessed : May 2025].

- [19] Alibaba Cloud. Alibaba cloud file storage NAS : Features, benefits and use cases. Alibaba Cloud Medium Blog, 2021. [Online]. Available : <https://alibaba-cloud.medium.com/alibaba-cloud-file-storage-nas-part-1-overview-and-explanation-3beb540125b4>. [Accessed : May 2025].
- [20] Open Container Initiative. OCI image format specification. GitHub, opencontainers/image-spec, 2024. [Online]. Available : <https://github.com/opencontainers/image-spec>. [Accessed : May 2025].
- [21] OpenGitOps. GitOps principles v1.0.0. OpenGitOps, CNCF Sandbox Project, 2021. [Online]. Available : <https://opengitops.dev>. [Accessed : May 2025].
- [22] NestJS. Authentication — NestJS documentation. NestJS Official Documentation, 2024. [Online]. Available : <https://docs.nestjs.com/security/authentication>. [Accessed : May 2025].
- [23] NestJS. JWT authentication — @nestjs/passport. DeepWiki, 2025. [Online]. Available : <https://deepwiki.com/nestjs/passport/4.1-jwt-authentication>. [Accessed : May 2025].
- [24] B. Jagdev. JWT authentication : A deep dive into access tokens and refresh tokens. Stackademic / Medium, 2023. [Online]. Available : <https://blog.stackademic.com/jwt-authentication-a-deep-dive-into-access-tokens-and-refresh-tokens-274c6c3b352c>. [Accessed : May 2025].
- [25] E. Duru. NestJS JWT authentication with refresh tokens complete guide. elvisduru.com, 2022. [Online]. Available : <https://www.elvisduru.com/blog/nestjs-jwt-authentication-refresh-token>. [Accessed : May 2025].
- [26] zenstok. How to implement refresh tokens with token rotation in NestJS. DEV Community, 2024. [Online]. Available : <https://dev.to/zenstok/how-to-implement-refresh-tokens-with-token-rotation-in-nestjs-1deg>. [Accessed : May 2025].
- [27] B. Jagdev. Refresh token rotation in NestJS — token security. Stackademic / Medium, 2023. [Online]. Available : <https://blog.stackademic.com/jwt-authentication-a-deep-dive-into-access-tokens-and-refresh-tokens-274c6c3b352c>. [Accessed : May 2025].
- [28] Auth.js Contributors. Refresh token rotation — Auth.js documentation. authjs.dev, 2024. [Online]. Available : <https://authjs.dev/guides/refresh-token-rotation>. [Accessed : May 2025].

- [29] M. Baranowski. Next.js authentication — JWT re-fresh token rotation with NextAuth.js. DEV Community, 2022. [Online]. Available : <https://dev.to/mabaranowski/nextjs-authentication-jwt-refresh-token-rotation-with-nextauthjs-5696>. [Accessed : May 2025].
- [30] Argo Workflows Contributors. REST API — argo workflows documentation. argo-workflows.readthedocs.io, 2024. [Online]. Available : <https://argo-workflows.readthedocs.io/en/latest/rest-api/>. [Accessed : May 2025].
- [31] Argo Workflows Contributors. Submit parameterized workflow template using REST API. GitHub Discussions, argoproj/argo-workflows, 2022. [Online]. Available : <https://github.com/argoproj/argo-workflows/discussions/8319>. [Accessed : May 2025].
- [32] NestJS. Server-sent events — NestJS documentation. NestJS Official Documentation, 2024. [Online]. Available : <https://docs.nestjs.com/techniques/server-sent-events>. [Accessed : May 2025].
- [33] G. Kumar. NestJS server-sent events (SSE) and its use cases. Medium, 2025. [Online]. Available : <https://medium.com/@kumar.gowtham/nestjs-server-sent-events-sse-and-its-use-cases-9f7316e78fa0>. [Accessed : May 2025].
- [34] Grafana Labs. Log queries — LogQL documentation. Grafana Loki Documentation, 2024. [Online]. Available : https://grafana.com/docs/loki/latest/query/log_queries/. [Accessed : May 2025].
- [35] Git SCM. Git clone — -depth option. Git Documentation, 2024. [Online]. Available : <https://git-scm.com/docs/git-clone>. [Accessed : May 2025].
- [36] Moby Project. BuildKit — concurrent, cache-efficient, and dockerfile-agnostic builder. GitHub, moby/buildkit, 2024. [Online]. Available : <https://github.com/moby/buildkit>. [Accessed : May 2025].
- [37] Uber Technologies. Zap — blazing fast, structured, leveled logging in go. GitHub, uber-go/zap, 2024. [Online]. Available : <https://github.com/uber-go/zap>. [Accessed : May 2025].
- [38] Railway. Railpack — zero-config application builder. GitHub, railwayapp/railpack, 2025. [Online]. Available : <https://github.com/railwayapp/railpack>. [Accessed : May 2025].

- [39] The Kubernetes Authors. Configure a security context for a pod or container. Kubernetes Documentation, 2024. [Online]. Available : <https://kubernetes.io/docs/tasks/configure-pod-container/security-context/>. [Accessed : May 2025].
- [40] SparkFabrik. Docker BuildKit deep dive : Optimize your build performance. SparkFabrik Tech Blog, 2025. [Online]. Available : <https://tech.sparkfabrik.com/en/blog/docker-cache-deep-dive/>. [Accessed : May 2025].
- [41] Helm Authors. Helm — the package manager for kubernetes. Helm Official Documentation, 2024. [Online]. Available : <https://helm.sh/docs/>. [Accessed : May 2025].
- [42] The Kubernetes Authors. Network policies. Kubernetes Documentation, 2024. [Online]. Available : <https://kubernetes.io/docs/concepts/services-networking/network-policies/>. [Accessed : May 2025].
- [43] Argo Workflows Contributors. Installation — argo workflows documentation. argo-workflows.readthedocs.io, 2024. [Online]. Available : <https://argo-workflows.readthedocs.io/en/latest/installation/>. [Accessed : May 2025].
- [44] Argo Workflows Contributors. Pod GC — argo workflows documentation. argo-workflows.readthedocs.io, 2024. [Online]. Available : <https://argo-workflows.readthedocs.io/en/latest/pod-gc/>. [Accessed : May 2025].